МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Факультет информационных технологий и прикладной математики

Кафедра вычислительной математики и программирования

Лабораторные работы по курсу «Информационный поиск»

Студент: К. А. Арусланов
Группа: М8О-403Б-23
Преподаватель: А. А. Кухтичев
Дата: 12.11.2026
Оценка:

Москва, 2026

Содержание

1 Лабораторная работа №1. Добыча корпуса документов 4
- 1.1 Постановка задачи . . . 4
- 1.2 Метод решения . . . 4
- 1.3 Результаты . . . 6
- 1.4 Вывод . . . 11

2 Лабораторная работа №2. Поисковый робот 12
- 2.1 Постановка задачи . . . 12
- 2.2 Метод решения . . . 12
- 2.3 Результаты . . . 13
- 2.4 Вывод . . . 15
- 2.5 Приложение А. Примеры запросов к MongoDB . . . 16

3 Лабораторная работа №3. Токенизация 17
- 3.1 Постановка задачи . . . 17
- 3.2 Метод решения . . . 17
- 3.3 Результаты . . . 19
- 3.4 Вывод . . . 24

4 Лабораторная работа №4. Лемматизация 25
- 4.1 Постановка задачи . . . 25
- 4.2 Метод решения . . . 25
- 4.3 Результаты . . . 27
- 4.4 Вывод . . . 33

5 Лабораторная работа №5. Закон Ципфа 34
- 5.1 Постановка задачи . . . 34
- 5.2 Метод решения . . . 34
- 5.3 Результаты . . . 36
- 5.4 Вывод . . . 38

6 Лабораторная работа №7. Булев индекс 39
- 6.1 Постановка задачи . . . 39
- 6.2 Метод решения . . . 39
- 6.3 Результаты . . . 40
- 6.4 Вывод . . . 42

7 Лабораторная работа №8. Булев поиск 43
- 7.1 Постановка задачи . . . 43
- 7.2 Метод решения . . . 43
- 7.3 План и результаты тестирования . . . 44
- 7.4 Результаты на корпусе . . . 45
- 7.5 Вывод . . . 47

8 Лабораторная работа №10. Координатный и цитатный поиск 48
- 8.1 Постановка задачи . . . 48
- 8.2 Метод решения . . . 48
- 8.3 План и результаты тестирования . . . 50
- 8.4 Результаты на корпусе . . . 50

8.5 Вывод . . . 52
9 Лабораторная работа №11. Сжатие индекса 54
9.1 Постановка задачи . . . 54
9.2 Метод решения . . . 54
9.3 Журнал выполнения . . . 55
9.4 План и результаты тестирования . . . 55
9.5 Результаты на корпусе . . . 56
9.6 Вывод . . . 59
10 Лабораторная работа №12. Указатели в индексе 60
10.1 Постановка задачи . . . 60
10.2 Метод решения . . . 60
10.3 План и результаты тестирования . . . 61
10.4 Результаты . . . 62
10.5 Вывод . . . 63
11 Лабораторная работа №13. Текстовое ранжирование TF-IDF 64
11.1 Постановка задачи . . . 64
11.2 Метод решения . . . 64
11.3 План и результаты тестирования . . . 66
11.4 Результаты . . . 66
11.5 Вывод . . . 70
12 Лабораторная работа №14. Сниппеты поисковой выдачи 72
12.1 Постановка задачи . . . 72
12.2 Метод решения . . . 72
12.3 План тестирования . . . 73
12.4 Результаты . . . 73
12.5 Вывод . . . 75
13 Лабораторная работа №17. Автотесты и модульное тестирование 76
13.1 Постановка задачи . . . 76
13.2 Метод решения . . . 76
13.3 План тестирования . . . 77
13.4 Результаты . . . 77
13.5 Вывод . . . 78
Исходный код 79

1 Постановка задачи

Необходимо проанализировать корпус документов, который будет использован при выполнении остальных лабораторных работ:

- Скачать примеры документов к себе на компьютер. В отчёте нужно указать источник данных. Источников в итоговом индексе должно быть не менее двух;
- Ознакомиться с ним, изучить его характеристики. Из чего состоит текст? Есть ли дополнительная мета-информация? Если разметка текста, какая она?
- Выделить текст.
- Найти существующие поисковики, которые уже можно использовать для поиска по выбранному набору документов (встроенный поиск Википедии, поиск Google с использованием ограничений на URL или на сайт). Если такого поиска найти невозможно, то использовать корпус для выполнения лабораторных работ нельзя!
- Привести несколько примеров запросов к существующим поисковикам, указать недостатки в полученной поисковой выдаче.

В результатах работы должна быть указаны статистическая информация о корпусе:

- Размер примеров «сырых» документов.
- Количество документов.
- Размер текста, выделенного из «сырых» данных.
- Средний размер документа, средний объём текста в документе.

2 Метод решения

2.1 Источники и хранение

В качестве тематики корпуса документов выбраны русскоязычные и англоязычные публикации о компьютерных играх. В итоговый список вошли восемь независимых игровых изданий. Русскоязычную часть составляют PlayGround.ru, StopGame.ru, IXBT.games, Игромания и GameMAG; англоязычную — PC Gamer, Eurogamer и GamingOnLinux.

Источник	Язык	Представленные типы публикаций
PlayGround.ru	русский	новости и пользовательские материалы
StopGame.ru	русский	новости, обзоры, статьи и блоги
IXBT.games	русский	новости, статьи и обзоры
Игромания	русский	новости, статьи и обзоры
GameMAG	русский	новости, статьи и обзоры
PC Gamer	английский	игровые новости, статьи и обзоры
Eurogamer	английский	новости, гайды, статьи и обзоры
GamingOnLinux	английский	новости и статьи об играх для Linux

Для анализа использованы 374 страницы восьми источников, сохранённые в MongoDB: 46 PlayGround, 51 StopGame, 46 IXBT.games, 36 Игромании, 54 GameMAG, 46 GamingOnLinux, 56 PC Gamer и 39 Eurogamer. Проверочные адреса перечислены в `urls.txt`, а расширенная выборка получена из sitemap. Полный инвентарь на этом этапе не обкачивался.

Сбор запускается командой:

```
python main.py collect urls.txt --delay 2
```

Перед запросом программа получает и сохраняет актуальный `robots.txt`, проверяет URL для собственного User-Agent и не запрашивает запрещённые пути. Ответ классифицируется как документ, редирект, 404, 429, 5xx, anti-bot/challenge или запрет robots.txt. CAPTCHA и WAF не обходятся.

При локальном сборе сырой HTML хранится в `data/raw/<source>/`, JSON с результатом разбора — в `data/parsed/<source>/`. Путь к исходному HTML хранится в JSON, поэтому после изменения парсера возможен повторный разбор без сети: `python main.py reparse`. В расширенной выборке HTML и результаты разбора сохранены в отдельных коллекциях MongoDB `raw_pages` и `documents`.

2.2 Оценка масштаба по sitemap

Перед массовой загрузкой для каждого источника были проверены правила `robots.txt` и объявленные в них sitemap. Программа `discover` обходит только разрешённые XML-карты, выделяет ссылки на публикации и сохраняет их отдельно от HTML. Для каждого сайта используются ограничения по стабильным частям URL или имени sitemap, чтобы не включать страницы тегов, авторов, изображений и служебные страницы. Затем команда `merge-inventories` объединяет локальные инвентари и удаляет совпадающие URL.

Таким образом можно получить число URL-кандидатов: до скачивания и разбора оно не учитывает возможные HTTP-ошибки, закрытые страницы и близкие по содержанию материалы.

2.3 Формат документа

Все парсеры возвращают источник, URL, заголовок, автора, дату публикации, категорию, теги, связанные игры, основной текст и исходные метаданные. Также записываются HTTP-статус, размеры сырого и выделенного текста, SHA-256 нормализованного текста и ошибка разбора. Удаляются только tracking-параметры URL, а содержательные параметры сохраняются.

2.4 Разметка и выделение текста

Проверка проведена на 374 сохранённых HTML. Основной текст выделен для 363 страниц; ещё 5 публикаций содержат только заголовок, а 6 страниц исключены из корпуса. Для каждого источника подтверждён селектор основного текста и корректность полей заголовка, автора, даты, тегов и категории. В обзорах GameMAG допускается несколько последовательных блоков основного текста; они объединяются в порядке следования.

Источник	Основной текст	Категория и таксономия
PlayGround	`div.article-content`	URL: `news` → news, `opinion` → review; JSON-LD `about.name` → games
StopGame	`article##material_content`	`newsdata` → news; `show` с «обзор» → review; теги и игры извлекаются из ссылок
IXBT.games	`div[id^="publication-"] .prose`	JSON-LD `articleSection`: новости, статьи, обзоры → news, article, review
Игромания	`div[class*="material-content_"]`	JSON-LD `@type`: `NewsArticle`, `Article`, `Review`; блок тегов → tags
GameMAG	`div.content-text`	URL: `news` → news, `reviews` → review; автор и теги — из блоков страницы
Gaming OnLinux	`article.h-entry .e-content`	JSON-LD `NewsArticle` → news; теги — только внутри статьи
PC Gamer	`div##article-body`	`mrf:tags`: `articleType` → news, review; виджеты удаляются
Eurogamer	`div.article_body_content`	JSON-LD `articleSection`; гайды и материалы → article

Для всех сайтов заголовок и служебные поля берутся из `h1`, JSON-LD или мета-тегов. У PlayGround внешний `article` включает комментарии и рекомендации. В IXBT.games числовая часть ID динамическая, а у Игромании динамичен суффикс CSS Modules, поэтому используются устойчивые фрагменты селекторов. Теги Игромании дублируются в desktop/mobile-блоках и дедуплицируются. В StopGame блоки `_read-more` ссылаются на другие материалы и удаляются до извлечения. У PC Gamer дополнительно исключаются блоки «поделиться», реклама, рекомендации, видео и форма рассылки.

3 Результаты

3.1 Потенциальный размер корпуса

По сохранённым sitemap сформирован объединённый инвентарь из **1 055 674 уникальных URL-кандидатов**; совпадающих URL между источниками не обнаружено. Это немного превышает целевой порог в один миллион документов. Наиболее крупные источники — GameMAG и Игромания, однако англоязычные PC Gamer и Eurogamer дают существенный запас объёма.

Источник	URL-кандидатов из sitemap
PlayGround.ru	100 000
StopGame.ru	128 309
IXBT.games	159 496
Игромания	190 628
GameMAG	191 303
PC Gamer	119 841
Eurogamer	137 934
GamingOnLinux	28 163
Всего	**1 055 674**

3.2 Статистика проверочной выборки

В MongoDB сохранены 374 HTML-страницы; статистика ниже рассчитана по ним на 8 октября 2026 года. Размер HTML получен из сохранённого поля `raw_pages.html` в кодировке UTF-8, размер текста — из `documents.parsed.text`.

Показатель	Значение
Сохранено HTML / пригодно для индекса	374 / 368
С текстом / только заголовок / исключено	363 / 5 / 6
Размер сохранённого HTML	170,00 МБ
Размер выделенного текста	1,41 МБ
Средний HTML / текст	454,5 / 3,8 КБ
Медиана текста	1,9 КБ
Минимум / максимум текста	0 / 67,4 КБ
Слов всего	157 261
Среднее / медиана слов	420,5 / 194,5
Точных дубликатов непустого текста	0
Коэффициент извлечения	0,8293%

Источник	Док.	HTML, МБ	Текст, КБ	Средний текст, КБ	Слов
PlayGround	46	7,77	131,0	2,8	10 152
StopGame	51	10,34	402,7	7,9	33 169
IXBT.games	46	17,83	138,8	3,0	11 191
Игромания	36	5,35	147,9	4,1	12 157
GameMAG	54	4,67	132,2	2,4	11 164
GamingOnLinux	46	3,14	77,0	1,7	13 104
PC Gamer	56	112,30	251,5	4,5	44 325
Eurogamer	39	8,59	128,8	3,3	21 999

3.3 Состояние корпуса после массовой загрузки

12 ноября 2026 года завершённый снимок, экспортированный из коллекции `documents`, содержит 798 999 пригодных публикаций восьми источников. Число URL-кандидатов исходного инвентаря не является числом сохранённых статей. В новом

снимке 626 875 новостей, 102 365 статей и 69 759 обзоров; 8 337 документов содержат только заголовок.

Показатель	Значение
Документов	798 999
Сырой HTML, сумма размеров до архивного сжатия	320,78 ГБ
Файлы архива сжатого HTML	70,83 ГБ
Заголовки и выделенный текст, UTF-8	2,45 ГБ
Средний сырой HTML	401,48 КБ
Средние заголовок и текст	3,06 КБ
Среднее число токенов в заголовке и тексте	335,37

Источник	Документы	Источник	Документы
GameMAG	137 571	IXBT.games	132 459
Eurogamer	125 250	PlayGround	104 696
StopGame	95 907	PC Gamer	88 757
Игромания	86 148	GamingOnLinux	28 211

Размер сырого HTML получен суммированием манифестов 14 архивных партий, а размер заголовков и текста — из манифеста неизменяемого поискового снимка. Это разные способы хранения тех же публикаций, поэтому размеры не складываются. КБ и ГБ здесь десятичные. Контрольная сумма снимка, числа по источникам и ограничения приведены в `corpus_refresh_2026-11-12.md`. Таблица проверочной выборки выше относится к 8 октября и сохраняется как исходный эксперимент.

3.4 Существующий поиск

Корпус доступен во встроенных поисках сайтов и во внешних поисковиках с ограничением `site:`. Для каждого кейса известен документ, который должен быть найден, поэтому недостатки выдачи проверяются сравнением результатов с сохранёнными HTML и выделенным текстом.

Запрос и ожидаемый документ	Наблюдение	Недостаток
«джо кросс»: новости о Джозефе Кроссе на StopGame и Игромании	Внутренний поиск StopGame выдаёт «кроссовер» и «кросс-плей». Яндекс с `site:stopgame.ru` не находит релевантную новость, а для Игромании показывает один документ. Google показывает релевантные публикации обоих сайтов в первых позициях.	Полнота зависит от поисковой системы; внутренний поиск StopGame плохо обрабатывает вариант имени «Джо/Джозеф Кросс».
«приземлённый подход Insomniac» из статьи IXBT.games о Marvel's Wolverine	Внутренний поиск IXBT.games сообщает «Ничего не найдено», хотя фраза содержится в статье. Яндекс с `site:ixbt.games` находит её первой.	Внутренний поиск не гарантирует поиск по полному тексту.

Запрос и ожидаемый документ	Наблюдение	Недостаток
«я играю роль поп-звезды, которая убила себя на сцене» из блога StopGame «Музыка Ночного Города»	Внутренний поиск StopGame возвращает несвязанные блоги; Яндекс выводит нужный блог первым.	Даже точная фраза не даёт ожидаемого результата; используемые токенизация и ранжирование непрозрачны.
«леман» на StopGame	Первый результат — «Косплемания: Chummomo...». Искомая последовательность присутствует лишь внутри слова «Косплемания», а не как сущность «Леман».	Сопоставление подстроки без границ слова даёт тематически нерелевантный результат.
Отбор по типу публикации	В просмотренных интерфейсах IXBT.games и Игромании нет группировки на новости, статьи и блоги; в StopGame есть вкладки типов и сортировка.	Часть внешних и встроенных поисков не использует поля корпуса `category`, `tags`, `games`; собственной системе нужны фильтры и зонный поиск.

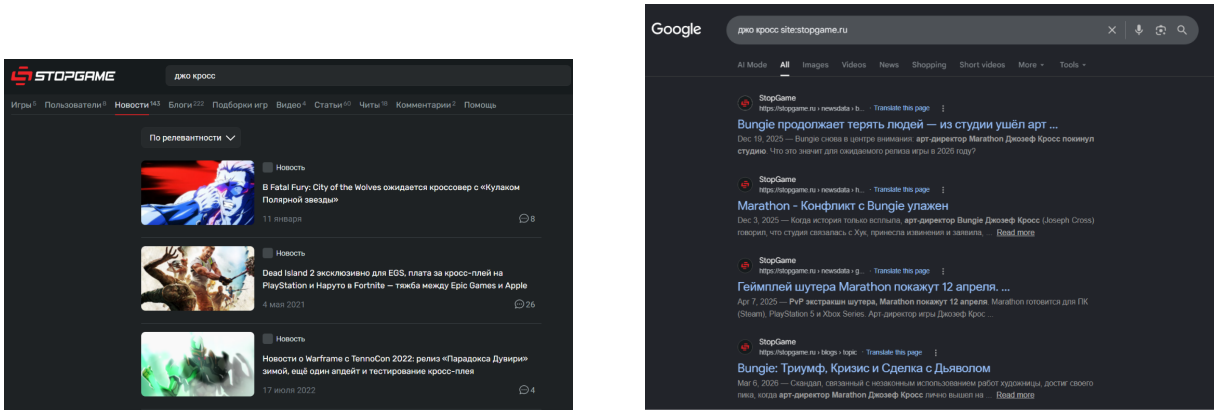


Рис. 1 — Запрос «джо кросс»: нерелевантные результаты встроенного поиска StopGame (слева) и релевантные результаты Google с ограничением на сайт (справа).

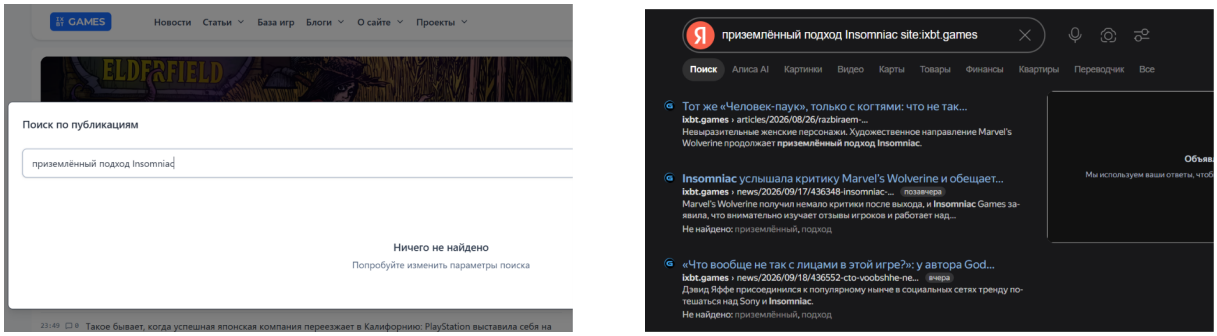


Рис. 2 — Точная фраза из статьи IXBT.games не находится внутренним поиском (слева), но находится Яндексом с `site:ixbt.games` (справа).

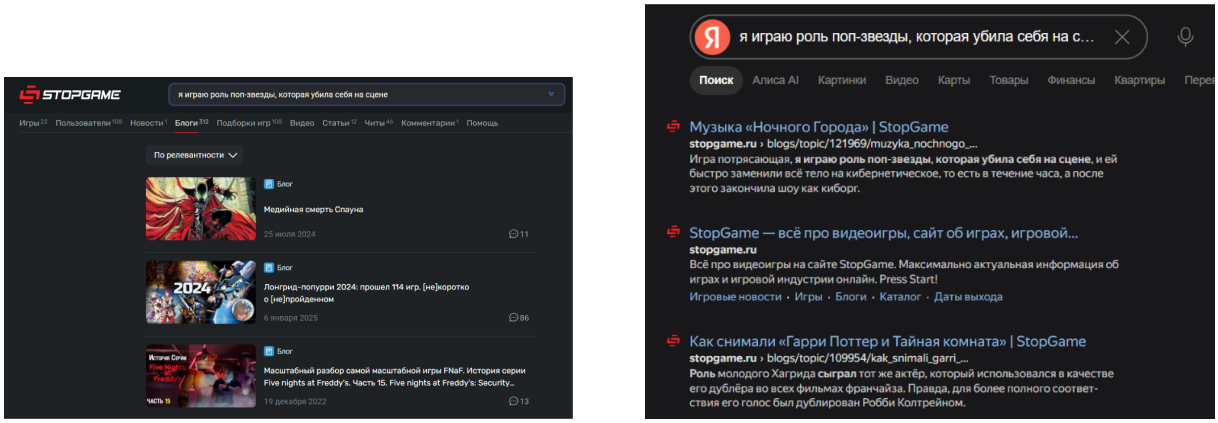


Рис. 3 — Поиск точной цитаты из блога StopGame: встроенный поиск возвращает другие блоги (слева), Яндекс — нужный документ первым (справа).

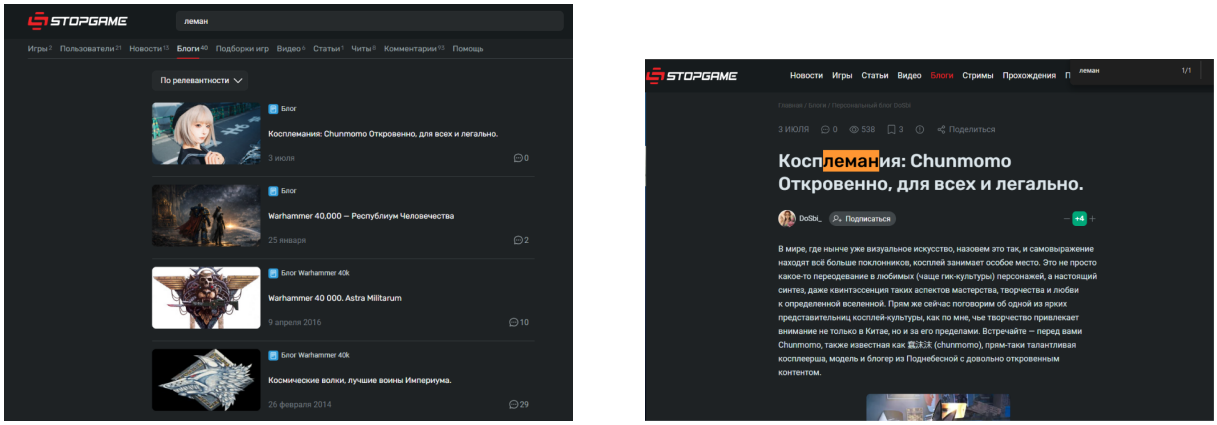


Рис. 4 — Ложное совпадение по подстроке «леман»: первый результат (слева) и контекст совпадения внутри слова «Косплемания» (справа).

4 Вывод

В ходе лабораторной работы я проанализировал структуру игровых публикаций на восьми сайтах и определил HTML-блоки, в которых находятся основной текст, заголовок, дата и другие полезные метаданные. Для этих источников были написаны парсеры, а результаты их работы сохранены в едином формате. Это позволило получить небольшую, но пригодную для дальнейших экспериментов проверочную выборку игрового корпуса.

Отдельно было интересно разобраться с правилами `robots.txt`: перед загрузкой страницы программа получает эти правила и не обращается к запрещённым путям. Проверка встроенного поиска сайтов, Google и Яндекса также показала, почему для корпуса понадобится собственный индекс: поисковая выдача не всегда полна, иногда содержит совпадения по подстроке и не даёт одинаково удобно отбирать новости, статьи и блоги.

1 Постановка задачи

Для корпуса игровых публикаций из первой лабораторной работы требуется разработать поисковый робот:

- реализовать обкачку документов;
- принимать при обычном запуске один аргумент — путь к YAML-файлу с данными базы в секции `db` и параметрами робота в секции `logic`, в том числе задержкой между запросами;
- сохранять в базе нормализованный URL, сырой HTML, название источника и дату загрузки в формате Unix timestamp;
- после остановки продолжать обход с незавершённого документа;
- периодически проверять загруженные страницы и обновлять сохранённые документы только при изменении содержимого.

2 Метод решения

2.1 Источники и входные данные

В качестве корпуса выбраны публикации восьми игровых сайтов. Русскоязычные источники — PlayGround, StopGame, IXBT.games, Игромания и GameMAG; англоязычные — PC Gamer, Eurogamer и GamingOnLinux. Входом служит UTF-8 TSV-файл: короткий `urls.txt` из первой лабораторной либо объединённый инвентарь sitemap. Последний содержит 1 055 674 URL-кандидата, но это ещё не число пригодных документов.

Файл читается построчно и импортируется в MongoDB партиями, без загрузки всего списка в память. URL очищается от параметров отслеживания, его хост сверяется с разрешённым списком. Пробная выборка воспроизводима по заданному seed; порядок очереди перемешивается устойчивым хешем URL.

2.2 Очередь и сохранение документов

Для хранения выбрана MongoDB. Очередь и содержимое страницы разделены по коллекциям, чтобы просмотр результатов разбора не загружал большой HTML:

Коллекция	Содержимое
`frontier`	URL, источник, состояние, время следующей попытки, число попыток, HTTP-заголовки и последняя ошибка
`raw_pages`	Нормализованный URL, источник, Unix-время загрузки, сырой HTML и SHA-256 ответа
`documents`	Результат разбора, HTTP-метаданные, время загрузки и ссылка `raw_page_id` на исходный HTML
`crawler_meta`	Метка импорта инвентаря и паузы для отдельных сайтов

Ключ `_id` у заданий и документов равен нормализованному URL, поэтому повторная загрузка не создаёт дубликатов. Сначала сохраняется сырой HTML, затем результат разбора и только после этого завершается задание очереди. Если выполнение прервётся между этими шагами, при следующей попытке согласованность записей проверяется по SHA-256.

Робот атомарно выбирает доступный URL и отмечает его как обрабатываемый. Затем он проверяет `robots.txt` и делает запрос с паузой для каждого хоста. Редиректы проверяются по отдельности: переход за пределы источника не выполняется. При перезапуске незавершённые URL возвращаются в начало очереди. Механизм рассчитан на один процесс робота.

2.3 Проверка изменений

При повторном запросе робот использует сохранённые `ETag` и `Last-Modified`. Ответ `304` оставляет HTML неизменным. Если сервер возвращает `200`, вычисляется SHA-256 исходного ответа: при совпадении хеша обе записи не перезаписываются. Для изменённого ответа обновляются `raw_pages`, `documents` и `fetched_at`. Если одна из двух записей отсутствует либо их хеши расходятся, робот повторно получает HTML и восстанавливает согласованное состояние. Время каждой проверки хранится в `frontier.last_checked_at`. При ошибках попытка откладывается: `404`/`410` и запрет `robots.txt` проверяются реже, для `429` учитывается `Retry-After`. Пауза одного источника хранится в MongoDB и не останавливает другие сайты. У завершённого задания время следующей проверки задаётся полем `available_at` через `revisit_seconds` после последней проверки. Обычный запуск берёт новые URL и повторы их временных ошибок. Повторная проверка выполняется отдельной командой `python revisit.py crawler.example.yaml`, только когда наступил назначенный срок. Эта команда не импортирует новые URL; после сбоя или остановки повторное задание остаётся за ней.

3 Результаты

3.1 Запуск и автотесты

Код робота запускается командой `python crawler.py crawler.example.yaml`. В примере ограничение `max_documents: 10` позволяет проверить небольшую часть списка `urls.txt`. Команда `python run_tests.py` проверяет систему без сервера MongoDB и без обращений к сайтам.

Успешно прошли 64 теста, в том числе 25 сценариев работы робота. Они охватывают сохранение HTML и времени Unix, продолжение после остановки, ответы `304` и `200`, контрольную сумму, `robots.txt` и `Retry-After`, а также отделение повторных проверок от первичной загрузки. Локальные HTML-образцы первой работы проверяются прежними тестами.

3.2 Пробная обкачка

MongoDB запускалась локально через Docker Compose. Сначала робот сохранил 24 проверочные страницы из `urls.txt`, затем выборка была расширена. На 8 октября 2026 года в очереди находилось 824 URL: 368 завершённых, 441 ожидаю-

щий, 14 исключённых и один оставшийся в обработке после остановки. Большой инвентарь в миллион URL в базу пока не импортировался.

Источник	Скачано	В корпусе	С текстом	Только заголовок
PlayGround	46	46	42	4
StopGame	51	51	51	0
IXBT.games	46	46	46	0
Игромания	36	36	36	0
GameMAG	54	54	53	1
PC Gamer	56	54	54	0
Eurogamer	39	36	36	0
GamingOnLinux	46	45	45	0
Всего	**374**	**368**	**363**	**5**

Все 374 записи `documents` имеют соответствующие страницы `raw_pages`; их контрольные суммы совпадают, пустого HTML и потерянных исходных страниц нет. Таблица отдельно показывает пригодные для индексации документы и страницы без основного текста, у которых можно использовать заголовок.

Первоначальные 124 записи были перенесены из формата со встроенным HTML в раздельные коллекции после резервного копирования базы. Средний размер записи в `documents` уменьшился примерно с 470 до 7 КБ, что удобнее для просмотра результатов разбора.

3.3 Состояние корпуса после массовой обкачки

12 ноября 2026 года из коллекции `documents` экспортирован новый неизменяемый снимок для поисковых лабораторных работ. В нём 798 999 пригодных публикаций восьми источников, на 320 166 больше предыдущего снимка из 478 833 документов. Суммарный объём заголовков и основного текста составляет 2 445 438 731 байт; 8 337 публикаций имеют только заголовок. Это статистика *снимка*, а не текущее состояние очереди: фильтр экспорта исключает непригодные для индексации записи.

Источник	Документов в снимке
IXBT.games	132 459
GameMAG	137 571
Eurogamer	125 250
PC Gamer	88 757
StopGame	95 907
Игромания	86 148
PlayGround	104 696
GamingOnLinux	28 211
Всего	**798 999**

Снимок и его SHA-256 описаны в `corpus_refresh_2026-11-12.md`. Числа пробной обкачки выше и ответы MongoDB в приложении относятся к 8 октября и не подменяются значениями нового снимка.

4 Вывод

В этой работе я дополнил сбор игровых публикаций устойчивой очередью в MongoDB. Теперь робот может продолжить обход после остановки и периодически проверять уже сохранённые страницы. Было полезно разобраться, как связать состояние очереди с сохранением документа и зачем при повторной проверке нужны `ETag`, `Last-Modified` и контрольная сумма ответа.

Я проверил робота тестами и пробной загрузкой в MongoDB. На практике выяснилось, что даже успешный ответ сайта не всегда означает пригодный текстовый документ: в sitemap встречаются страницы категорий, а у некоторых публикаций иная разметка. При последующей массовой обкачке корпус вырос до 798 999 пригодных публикаций; доля непригодных страниц и расход места остаются важными для дальнейших запусков.

Приложение А. Примеры запросов к MongoDB

Ниже приведены читающие запросы к базе `mai_ir` и ответы на них после пробной обкачки 8 октября 2026 года. При продолжении загрузки числа изменятся.

А.1 Состояние очереди

```
db.frontier.aggregate([
  { $group: { _id: "$state", count: { $sum: 1 } } },
  { $sort: { _id: 1 } }
])
```

Ответ: `done: 368`, `excluded: 14`, `in_progress: 1`, `pending: 441`.

А.2 Интервал до повторной проверки

```
db.frontier.aggregate([
  { $match: { state: "done" } },
  { $group: {
      _id: { $subtract: ["$available_at", "$last_checked_at"] },
      count: { $sum: 1 }
  } }
])
```

Ответ: `[{_id: 2592000, count: 368}]`. Таким образом, для всех завершённых заданий следующая проверка назначена через 2 592 000 секунд, то есть через 30 дней после предыдущей.

А.3 Связь разобранных документов с HTML

```
db.documents.aggregate([
  { $lookup: {
      from: "raw_pages", localField: "raw_page_id",
      foreignField: "_id", as: "raw"
  } },
  { $unwind: "$raw" },
  { $match: { $expr: {
      $eq: ["$content_sha256",
            "$raw.content_sha256"]
  } } },
  { $count: "matched" }
])
```

Ответ: `[{matched: 374}]`. Каждый из 374 разобранных документов нашёл соответствующий сырой HTML с тем же хешем.

А.4 Документы без выделенного текста

```
db.documents.countDocuments(
  { "parsed.parse_error": { $type: "string" } })
```

Ответ: `6`. Все шесть записей исключены из будущего поискового индекса; у пригодных документов ошибок разбора нет.

1 Постановка задачи

Нужно реализовать процесс разбиения текстов документов на токены, который потом будет использоваться при индексации. Для этого потребуется выработать правила, по которым текст делится на токены. Необходимо описать их в отчёте, указать достоинства и недостатки выбранного метода. Привести примеры токенов, которые были выделены неудачно, объяснить, как можно было бы поправить правила, чтобы исправить найденные проблемы.

В результатах выполнения работы нужно указать следующие статистические данные:

- количество токенов;
- среднюю длину токена.

Кроме того, нужно привести время выполнения программы, указать зависимость времени от объёма входных данных. Указать скорость токенизации в расчёте на килобайт входного текста. Является ли эта скорость оптимальной? Как её можно ускорить?

2 Метод решения

2.1 Корпус и подготовка данных

Темой корпуса являются русскоязычные и англоязычные публикации о компьютерных играх восьми источников. Входные данные — публикации, сохранённые роботом второй работы в коллекции `documents` базы MongoDB. Используются только пригодные для индексации записи без ошибки парсера. Из каждой записи экспортируются заголовок и основной текст. Публикации без основного текста участвуют заголовком; страницы-каталоги исключаются до токенизации.

Python-экспортёр создаёт неизменяемый локальный снимок с бинарным файлом `corpus.bin`, таблицей соответствия ID и URL `documents.jsonl` и описанием `manifest.json`. Для снимка сохраняется SHA-256. В бинарном формате поля предваряются длинами в байтах, поэтому переводы строк и нулевые символы не нарушают границы документов. Заголовок и основной текст остаются отдельными полями. Сырой HTML при этом не читается.

Экспорт идёт курсором MongoDB, обработка — по одному документу. Весь корпус не требуется помещать в оперативную память. Снимок позволяет повторять измерения на одинаковых данных и исключить MongoDB из замеров производительности токенизатора. Экспорт не меняет базу. В описании снимка зафиксированы начало и конец экспорта. Обычный курсор не создаёт транзакционный снимок MongoDB: при одновременной обкачке состав базы может меняться. Все приведённые подсчёты относятся к завершённому локальному файлу.

2.2 Алгоритм и правила

Основная программа написана на C++17 с использованием разрешённой для токенизации STL. Алгоритм самостоятельно декодирует UTF-8 и последовательно просматривает кодовые точки. Версия правил обозначена `gaming-v3`.

– Буквы ASCII, русские буквы, латиница Latin-1 и Extended-A, а также ASCII-цифры входят в состав слов. Цифры сохраняются и внутри названий: `PS5` превращается в `ps5`, `3D` — в `3d`.

– Регистр нормализуется по явным таблицам поддерживаемых букв, независимо от локали системы. Буква «ё» не заменяется на «е».

– Внутренние дефисы и апострофы сохраняются. Типографские апострофы приводятся к ASCII-апострофу, обычный и неразрывный дефисы — к `-`. Соединитель допустим только внутри слова перед буквой или цифрой. Среднее и длинное тире считаются разделителями.

– Обозначения `C++`, C# и их варианты с номером стандарта, например `C++17`, сохраняются целиком. Правило применяется к отдельному `C`/`c`, а не к любой последовательности букв перед плюсом.

– В числовой последовательности точка сохраняется перед следующей цифрой: `3.0.1` становится одним токеном. Поддерживается префикс `v`, например `V1.2.3` превращается в `v1.2.3`. Конечная точка предложения отделяется. Это лексическое правило: десятичные числа и IP-адреса также сохраняются целиком, без определения типа сущности.

– Цепочка из трёх и более одиночных букв через точки превращается в один токен без точек: `S.T.A.L.K.E.R.` и `STALKER` дают одинаковый терм `stalker`. Конечная точка необязательна. Две буквы через точку не склеиваются, многобуквенные сегменты не распознаются как такая цепочка. Правило действует на поддерживаемую латиницу и русские буквы, без словаря названий игр.

– Пробелы, переводы строк, остальная пунктуация и emoji разделяют слова. Стоп-слова, короткие слова и числа не удаляются.

– Комбинируемые знаки U+0300–U+036F присоединяются к начатому токену. Unicode-нормализация NFC/NFKC не выполняется. Остальные письменности пока считаются разделителями; полное Unicode case folding не реализовано.

– Повреждённый UTF-8 и обрезанный бинарный формат считаются ошибкой входных данных. Программа завершается с ненулевым кодом возврата.

Каждый токен содержит ID документа, имя поля, позицию с нуля внутри поля, начальное и конечное байтовые смещения и нормализованный терм. Смещения относятся к исходному тексту, а не к строке после нормализации. Такое представление можно использовать при построении позиционного индекса и сниппетов. Тот же модуль подходит для обработки поисковых запросов. Лемматизация и стемминг в этой работе не выполняются.

Просмотр занимает линейное время относительно числа байтов текста. В памяти остаются текущий документ и буфер токена; глобальный словарь терминов не строится. Длина токена считается в кодовых точках нормализованной строки, включая внутренние соединители, а не в байтах UTF-8. Это также не число видимых графем: буква и отдельный комбинируемый знак дают две кодовые точки. Средняя длина равна отношению суммы длин токенов к их количеству; при отсутствии токенов принимается равной нулю.

2.3 План тестирования и измерений

Модульные тесты на C++ с GoogleTest вызывают токенизатор напрямую и проверяют русский и английский текст, регистр и «ё», цифры, типографские соединители, обозначения языков программирования, версии, цепочки одиночных букв через точки, диакритику, границы UTF-8, позиции внутри полей, пустые и большие тексты. Тесты обработчика командной строки проверяют вывод TSV и JSON-статистику. Отдельно проверяются повреждённая кодировка, обрезанные записи, недопустимые длины, неверные аргументы, сохранение существующих файлов и отсутствие опубликованного частичного результата после ошибки. CTest также запускает сам исполняемый файл для проверки справки и обработки неизвестного режима. Тесты экспортёра проверяют отбор документов, сохранение публикаций с одним заголовком и отсутствие изменений в базе. Тесты демонстрации проверяют соответствие фрагмента исходной статье, байтовые границы, выбор документа и вывод UTF-8.

Для измерения зависимости времени от размера берутся каждый восьмой, четвёртый, второй документ снимка и весь снимок. Подвыборки распределены по исходному порядку URL, поэтому включают разные сайты. Тексты не дублируются для искусственного увеличения объёма. На каждой подвыборке проводится один прогрев и 5 измерений; основным результатом служит медиана. Все повторы сохраняются в `benchmark.json`.

Отдельно измеряется сканирование с нормализацией и счётчиками, а также полное чтение и обработка бинарного файла. В этих повторах запись токенов выключена. Ещё один запуск сохраняет полный TSV и показывает стоимость вывода. Создание процесса и экспорт MongoDB в приведённое внутреннее время не входят. Файловый кеш прогрет; результаты не характеризуют чтение с холодного диска. Везде используется десятичный килобайт: 1 КБ = 1000 байт.

3 Результаты

3.1 Количество и длина токенов

Эксперимент выполнен 21 октября 2026 года на снимке из 12 497 публикаций, из них 121 содержат только заголовок. Суммарный размер заголовков и основного текста — 40 324,69 КБ. Статистика рассчитывается по обоим полям без удаления стоп-слов и словоформ.

Показатель	Значение
Количество документов	12 497
Количество токенов	4 140 679
Средняя длина токена, кодовых точек	5,308

Источник	Документы	Токены	Средняя длина
Eurogamer	1058	521 285	4,682
GameMAG	2442	482 025	5,700
GamingOnLinux	378	121 603	4,734
Игромания	2411	731 247	5,734
IXBT.games	1817	338 056	5,880
PC Gamer	1477	888 458	4,625
PlayGround	1290	215 523	5,858
StopGame	1624	842 482	5,537

Статистика получена по всем пригодным записям завершённого экспорта. Объём пока не достигает требуемых курсом 30–50 тысяч документов. После продолжения обкачки необходимо повторить экспорт и тот же эксперимент. Копии текстов для увеличения числа документов не добавлялись.

3.2 Производительность

Программа собрана с оптимизацией `-O3` и стандартом C++17 на Windows AMD64. Точная версия компилятора, его флаги и SHA-256 бинарного файла сохранены вместе с результатами. Объём в таблице включает только UTF-8 заголовки и тексты, без служебных полей бинарного формата.

Документы	Текст, КБ	Обработка, мс	С чтением, мс	Скорость, КБ/с
1563	5 147,28	51,415	57,767	89 104,9
3125	9 682,72	100,796	114,002	84 934,6
6249	19 961,70	199,960	224,238	89 020,3
12497	40 324,69	389,638	431,545	93 442,7

На всём снимке медиана времени обработки составляет 389,638 мс, с чтением — 431,545 мс. Скорость чтения и обработки равна 93 442,7 КБ/с, или 0,0107 мс/КБ. Запуск с сохранением полного TSV занимает 1 581,327 мс: форматирование и запись результата добавляют работу, которой нет в измерениях без вывода. По внешнему таймеру медиана полного запуска без TSV составляет 441,188 мс, а отдельный запуск с TSV — 1 594,473 мс. Эти значения дополнительно включают создание процесса и передачу его небольшого JSON-ответа вызывающему Python-скрипту.

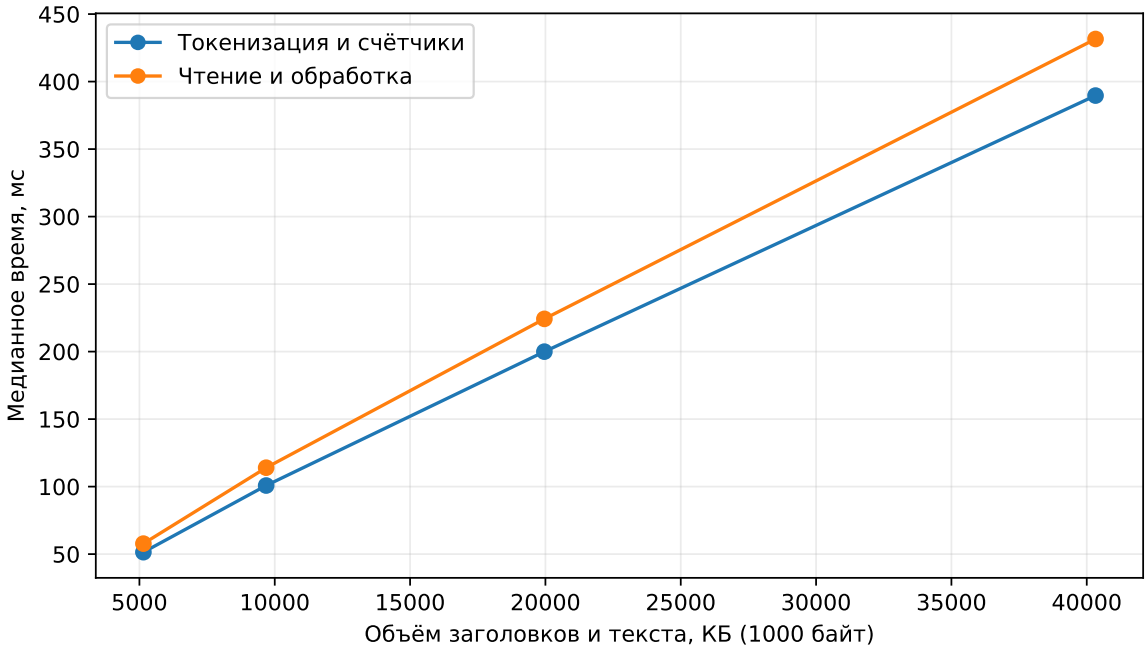


Рис. 5 — Зависимость медианного времени от объёма входного текста.

Линейный проход соответствует ожидаемой зависимости порядка $O(B)$, где B — объём текста. На небольших подвыборках также влияют накладные расходы, состав русских и английских текстов и планирование процессов. Четырёх измеряемых объёмов недостаточно для вывода о скорости на миллионном корпусе.

По порядку сложности полный просмотр текста улучшить нельзя: нужно прочитать входные символы. При этом измеренная реализация не доказана оптимальной по постоянным затратам. Возможные ускорения — отдельный быстрый путь для ASCII, уменьшение расходов вызова обработчика токенов и пакетная запись более компактного результата. Параллельная обработка независимых документов также возможна; сравнивать её следует отдельным экспериментом с теми же правилами и проверкой одинакового результата.

3.3 Повторный прогон на пополненном корпусе

12 ноября 2026 года экспортирован новый снимок из 798 999 пригодных публикаций. Заголовки и основной текст занимают 2 445 438 731 байт; 8 337 публикаций имеют только заголовок. Тот же токенизатор `gaming-v3` выделил 267 962 986 токенов со средней длиной 5,226 кодовой точки. Число токенов совпало с независимо выполненным подсчётом частот для ЛР5. Данные о снимке и его SHA-256 сохранены в `corpus_refresh_2026-11-12.md`.

Источник	Документы	Токены
GameMAG	137 571	25 041 346
IXBT.games	132 459	24 079 997
Eurogamer	125 250	63 715 251
PlayGround	104 696	18 737 129
StopGame	95 907	47 381 713
PC Gamer	88 757	52 955 801
Игромания	86 148	26 230 469
GamingOnLinux	28 211	9 821 280
Всего	**798 999**	**267 962 986**

Один повторный запуск без вывода TSV на этом снимке дал 38 589 мс собственно обработки и 67 006 мс вместе с чтением файла с диска E:. Скорость чтения и обработки составила 36 496 КБ/с, или 0,0274 мс/КБ. Это одиночное измерение; медианы, график и таблица подвыборок выше относятся к исходному снимку из 12 497 документов. Поэтому разницу времени нельзя приписывать только увеличению корпуса: поменялись носитель и состояние файлового кеша. Полный JSON нового прогона сохранён в `lab3/refresh_2026-11-12.json`.

3.4 Качество выделения и ограничения

В сохранённых публикациях найдены следующие характерные случаи. Короткие фрагменты и ожидаемый результат включены в тесты GoogleTest.

Фрагмент	Результат	Наблюдение и возможное улучшение
`C++`, C# (PC Gamer)	`c++`, `c#`	В первой версии оба обозначения превращались в `c`. Добавлено правило сохранения специальных суффиксов; теперь три названия различаются.
`3.0.1` (GameMAG)	`3.0.1`	В первой версии получались три числа. Теперь точка между цифрами сохраняется, а конечная точка предложения отделяется.
`S.T.A.L.K.E.R.`	`stalker`	Цепочка одиночных букв теперь распознаётся целиком. Точки удаляются, поэтому написание `STALKER` даёт тот же терм.
`Half-Life` (GameMAG)	`half-life`	Название сохраняется, но запрос `Half Life` даёт два токена. Для сопоставления вариантов понадобится согласованное правило альтернативного представления.
`Pokémon` (IXBT.games)	`pokémon`	Диакритика сохраняется; `Pokemon` будет другим термом. Возможен отдельный вариант без диакритики, но его следует оценить на неоднозначных названиях.

Пример с C++ и C# взят из публикации PC Gamer, версия — из новости GameMAG о Destruction AllStars. Положительные примеры — новость GameMAG о Black Mesa и материал IXBT.games о патенте Nintendo.

Название `S.T.A.L.K.E.R.` встречается и в сохранённой публикации GameMAG о КРИ Awards. Его нормализация демонстрируется на исходном фрагменте этой ста-

тьи. Обратная сторона общего правила — `a.b.c` также даст `abc`, даже если это инициалы или части доменного имени. Для различения таких случаев понадобились бы дополнительные ограничения или контекст. Другие ограничения — неполный набор письменностей и отсутствие NFC. Новые правила нужно применять одинаково к документам и запросам, меняя версию токенизатора.

3.5 Автотесты и воспроизведение

Число успешно выполненных автоматических проверок 22 октября 2026 года составило 43. Из них 34 запускаются через CTest: 17 тестов правил токенизации, 9 тестов обработчика командной строки и 6 тестов демонстрации написаны на C++ с GoogleTest; ещё 2 проверки запускают сам исполняемый файл. На Python остаются 9 проверок вспомогательных компонентов: 4 теста экспортёра и 5 тестов совместимости прежней демонстрации. Алгоритм токенизации в Python-тестах не дублируется.

Часть тестов содержит несколько входных случаев. Полный журнал приложен как `doc/lab3/tests.txt`; данные эксперимента находятся в `statistics.json`, `benchmark.json` и `benchmark.csv` того же каталога. Для тестов подготовлен отдельный Docker-образ с компилятором, CMake и GoogleTest. После однократной сборки образа проверки работают без сети и MongoDB. Исходники подключаются только для чтения, результаты сборки сохраняются в отдельном томе:

```
docker compose --profile tests build tokenizer-tests
docker compose run --rm --no-deps tokenizer-tests
```

Полный эксперимент (сборка, оба набора проверок, экспорт и замеры) запускается одной командой:

```
python run_lab3.py --config crawler.large.yaml \
  --output data/tokenization/run_001 --report-dir doc/lab3
```

В примере перенос строки дан для оболочки Bash; команду также можно ввести одной строкой. Требования и отдельные команды описаны в `doc/lab3_run.md`. Снимки корпуса и полные токены хранятся локально в `data/tokenization/` и не включаются в Git.

Демонстрация также написана на C++ и использует тот же модуль токенизации напрямую. После сборки её можно запустить командой:

```
docker compose run --rm --no-deps tokenizer-tests tokenizer_demo
```

Она выбирает реальные статьи из последнего локального снимка и выводит источник, ID, заголовок, URL, исходный фрагмент и таблицу токенов. Колонка `original` извлекается из исходного текста по байтовым границам, возвращённым C++-программой. Позиции и смещения относятся к целому полю статьи, а не к началу показанного фрагмента. Демонстрируются версии, обозначения языков и нормализация названий с точками. Произвольный документ можно выбрать опцией `-document-id`, фрагмент — опцией `-find`.

4 Вывод

В этой работе я реализовал токенизацию игровых публикаций и проверил её на русских и английских текстах. Было интересно увидеть, что даже простое разбиение на слова требует нескольких решений: сохранять ли дефис, как обрабатывать апострофы и что делать с цифрами в названиях игр. Особенно наглядными оказались C++ и C#: сначала после удаления пунктуации они превращались в одинаковый токен. Я уточнил правила, и теперь эти обозначения, как и номера версий игр, сохраняются целиком. Для названий вроде `S.T.A.L.K.E.R.` я добавил объединение одиночных букв через точки: теперь их можно сопоставлять с обычным написанием без точек.

Я получил статистику токенов, измерил скорость обработки и подготовил повторяемый запуск экспериментов и демонстрацию на настоящих статьях. Позиции и исходные границы слов сохранены, поэтому результат можно использовать в следующих работах. Повторный запуск на 798 999 документах подтвердил число токенов, полученное при подсчёте частот. Обработка словоформ выполнена отдельно в ЛР4.

1 Постановка задачи

Добавить в созданную поисковую систему лемматизацию. В простейшем случае, это просто поиск без учёта словоформ. В более сложном случае, можно давать бонус большего размера за точное совпадение слов.

Лемматизацию можно добавлять на этапе индексации, можно на этапе выполнения поискового запроса.

В отчёте должна быть включена оценка качества поиска, после внедрения лемматизации. Стало ли лучше? Изучите запросы, где качество ухудшилось. Объясните причину ухудшения и как можно было бы улучшить качество поиска по этим запросам, не ухудшая остальные запросы?

2 Метод решения

2.1 Корпус и словарные данные

Темой корпуса выбраны русскоязычные и англоязычные публикации о компьютерных играх восьми сайтов. Из MongoDB экспортирован неизменяемый локальный снимок: 27 014 документов, 85,42 МБ заголовков и основного текста в UTF-8. Из них 263 публикации участвуют только заголовком. В этой работе коллекции робота не изменяются: леммы являются производными данными для поиска. SHA-256 снимка, словаря и исполняемых файлов сохранены вместе с результатами. Дополнительно лемматизация проверена в булевом поиске на снимке из 478 833 документов, использованном в ЛР7–ЛР8. В обоих экспериментах сравниваются режимы на одном и том же наборе документов; результаты разных снимков не объединяются в одну оценку качества.

Для русского языка используются словоформы и парадигмы OpenCorpora из пакета `pymorphy3-dicts-ru` версии `2.4.417150.4580142`. Это ревизия 417150, скомпилированная 8 января 2022 года. Python-скрипт читает формат фиксированного пакета словарных данных через `DAWG2-Python`; готовый морфологический анализатор `pymorphy3` не используется. Нормальная форма восстанавливается из сохранённых частей парадигмы: префикса, основы и суффикса.

Для английского языка используются списки основ по частям речи и исключения WordNet 3.0. Скрипт подготовки потоково читает данные, удаляет повторные пары через временную SQLite и записывает собственный бинарный формат. Версии, контрольные суммы, сведения об источниках и лицензиях записываются в `manifest.json` и `NOTICE.txt` рядом со словарём. Это использование словарных данных, а не библиотечного алгоритма поиска.

Источники данных: пакет русских словарей, WordNet 3.0.

2.2 Собственный анализатор на C++

Токенизатор предыдущей работы сначала выделяет слова, приводит регистр и нормализует точечные сокращения. Затем лемматизатор получает нормализованный токен и возвращает одну или несколько лемм. Исходный терм, позиция и байтовый диапазон не теряются. Документ и запрос обрабатываются одинаковыми правилами.

Основные компоненты лемматизации и экспериментального поиска написаны на C++17 без контейнеров STL. Используются собственные массивы, байтовый пул

и хеш-таблица с открытой адресацией и линейным пробированием. Плотность таблицы не превышает 50%; хеш — FNV-1a. Словарь загружается один раз на объект, а результаты ссылаются на уже размещённые данные. Выделения памяти для каждого словарного токена не требуется. Нормально ожидаемая стоимость поиска пропорциональна длине ключа с учётом пробирований; это не гарантия постоянного времени в худшем случае. STL остаётся в разрешённом условием токенизаторе и тестовом коде, Python используется для подготовки и измерений.

Русские словоформы ищутся непосредственно в таблице. Несколько грамматических разборов с одной леммой объединяются; разные леммы сохраняются: «стали» → «сталь | стать». Контекстное снятие омонимии не выполняется. Для английских слов проверяются исключения и собственные суффиксные правила по частям речи. Полученная основа принимается только при наличии в словаре соответствующей части речи. Это не полная реализация WordNet Morphy с обработкой словосочетаний.

Неизвестное слово остаётся исходным токеном. Анализатор не переводит слова, не подбирает синонимы и не заменяет слово произвольным похожим корнем. Буквы «е» и «ё» различаются. Названия `PS5`, `C++`, `C##`, версии `3.0.1`, слова с дефисами и апострофами сохраняются после нормализации токенизатора. Например, `S.T.A.L.K.E.R.` уже поступает как `stalker`. Новые названия и сленг поэтому не исчезают из поиска.

При загрузке проверяются сигнатура формата, размеры, границы смещений, UTF-8, контрольная сумма и повторные ключи. Максимум вариантов в интерфейсе равен 64: превышение вызывает явную ошибку, а не незаметное усечение. В подготовленном словаре максимальная неоднозначность равна пяти. Файл занимает 132,99 МБ; словарь с таблицей в памяти — 166,54 МБ без буферов текущего текста. Весь корпус в RAM не загружается: обработка идёт последовательно по полям документов.

2.3 Сравнение поиска до и после

Для проверки эффекта реализована утилита `lemma_search`. Это последовательный диагностический поиск по снимку, не готовый инвертированный индекс и не TF-IDF. Она поддерживает заголовки и основной текст; в эксперименте качества используются только заголовки.

Сравниваются три режима на одинаковых документах и запросах:

- `exact`: совпадение исходных нормализованных токенов;
- `lemma`: совпадение хотя бы одной леммы каждого слова запроса;
- `hybrid`: те же кандидаты, что у `lemma`, с бонусом за точные совпадения исходных токенов.

Между различными словами запроса действует AND, между вариантами одного слова — OR. Повторные одинаковые слова запроса удаляются. Базовая оценка во всех режимах равна $h(m+1)$, где h — число покрытых в заголовке слов запроса, m — число различных слов запроса. В гибридном режиме прибавляется число точных совпадений, от 0 до m. Бонус меняет порядок только внутри группы с одинаковым покрытием заголовка. Оставшиеся равенства разрешаются по возрастанию ID документа; такой порядок детерминирован, но сам по себе не отражает релевантность.

До сравнения зафиксирован диагностический набор: 37 реальных заголовков и 12 запросов. Для каждой пары запрос–документ задана бинарная релевантность; неразмеченных документов в этом наборе нет. Примеры специально подобраны для проверки словоформ, названий и омонимии, а не случайно выбраны из корпуса. Разметка предварительная авторская, на основе заголовков и просмотренных фрагментов; независимой экспертной оценки не проводилось. Поэтому полученные значения характеризуют диагностический набор и не переносятся на весь корпус.

Используются $P@5$, $Recall@5$ и $nDCG@5$:

$$P@5 = \frac{N_5}{5}, \qquad Recall@5 = \frac{N_5}{R},$$

где N_5 — число релевантных документов в первых пяти результатах, R — число релевантных документов именно в размеченном наборе. Если результатов меньше пяти, знаменатель точности не уменьшается.

$$DCG@5 = \sum_{i=1}^{5} \frac{2^{r_i} - 1}{\log_2(i+1)}, \qquad nDCG@5 = \frac{DCG@5}{IDCG@5}.$$

Идеальный порядок строится по известным оценкам того же набора; при нулевом идеальном значении метрика определена как ноль. Итог — среднее по запросам с одинаковыми весами. Python считает метрики по выдаче C++ и не участвует в выборе кандидатов или их ранжировании.

2.4 Тестирование и измерения

План тестирования включает словарные словоформы и омонимию, английские исключения и суффиксы, неизвестные игровые названия, защищённые токены, коллизии и повреждения бинарного формата. Проверяются время жизни ссылок на неизвестные слова, границы UTF-8 и отсутствие частичной выдачи при ошибке входного корпуса. Тесты поиска покрывают AND, повторы слов, режимы полей, стабильность top-k, ухудшение по Windows и исправление бонусом. Для расчёта метрик используются небольшие примеры с заранее вычислимым ответом.

GoogleTest и процессные проверки запускаются через CMake/CTest в Docker. Успешно выполнено 66 проверок общего C++ набора ЛР3–ЛР4 и 17 Python-тестов подготовки словаря и расчёта метрик. Журнал с отдельными тесткейсами приложен в `lab4/tests.txt`.

Производительность измерена для Windows Release-сборки GCC 13.1.0 на одном и том же локальном снимке. Для префиксов 1/8, 1/4, 1/2 и всего корпуса выполнены прогрев и 5 повторов; приводятся медианы. Время обработки включает токенизацию и поиск лемм, но не запись TSV, чтение корпуса и загрузку словаря. Последние измеряются отдельно. Измерения используют прогретый файловый кеш и не моделируют холодный запуск. Состав сайтов и языков у префиксов различается, поэтому рост времени не следует считать доказательством строгой линейности.

3 Результаты

3.1 Примеры соответствия токенов и лемм

Для наглядной проверки добавлен режим `lemmatizer demo`. Он вызывает тот же токенизатор и лемматизатор, что обработка корпуса, и выводит исходное написа-

ние, нормализованный токен, признак словарного разбора и все варианты лемм. Без `-input` используется небольшая демонстрационная фраза; с этим параметром можно разобрать собственный UTF-8 файл.

Исходное	Токен	Лемма или леммы	Словарь
Игроки	игроки	игрок	да
играли	играли	играть	да
в	в	в	да
игры	игры	игра	да
Люди	люди	человек	да
ёлки	ёлки	ёлка	да
стали	стали	сталь | стать	да
Games	games	game	да
were	were	be	да
better	better	better | good | well	да
C++	c++	c++	нет
C#	c#	c#	нет
PS5	ps5	ps5	нет
3.0.1	3.0.1	3.0.1	нет
Джозеф	джозеф	джозеф	да
Джозефа	джозефа	джозеф	да
Джозефу	джозефу	джозеф	да
Джо	джо	джо	да

Результат реального запуска C++ с двуязычным словарём

Таблица автоматически получена из машинного TSV, а не заполнена ожидаемыми ответами вручную. Вход находится в `tests/fixtures/lab4_demo.txt`, полная таблица — в `lab4/token_demo.txt`; хеши словаря, бинарника и входа сохранены в `lab4/token_demo.json`. Позиции и байтовые границы можно проверить в `lab4/token_examples.tsv`. Отметка «нет» означает сохранение исходного токена; слово при этом не теряется.

Пример «стали» показывает неоднозначность: возвращаются «сталь» и «стать» на одной позиции. Для `better` также сохраняются несколько словарных вариантов. Это не контекстный выбор правильного значения.

Имена «Джозефа» и «Джозефу» приводятся к «джозеф», а «Джо» остаётся «джо». Здесь словарь уже содержит нужные русские формы: отсутствие перехода «Джозеф» → «Джо» не является недостатком лемматизации. Сокращённые имена относятся к синонимам и требуют отдельного расширения запроса с учётом контекста; безусловное объединение может добавить ложные совпадения. Морфологический словарь в этом эксперименте не менялся.

Автотесты дополнительно проверяют соответствие таблицы машинному TSV и различение словоформ имени и сокращённых имён.

3.2 Обработка корпуса

Покрытие словарём не является точностью лемматизации: наличие разбора не гарантирует его соответствие контексту. Число исходных токенов также не равно числу вариантов лемм. На полном снимке медиана обработки составляет 6 023,36 мс,

Показатель	Значение
Документов	27 014
Заголовки и текст, МБ	85,42
Исходных токенов	9 020 624
Доля токенов со словарным разбором, %	80,27
Токенов с изменённой формой	3 230 212
Токенов с несколькими вариантами	1 003 108
Ключей в словаре	3 111 758

или 1 497 607 токенов/с; с чтением корпуса — 6 176,49 мс. Загрузка словаря занимает отдельно 1 046,85 мс. Для долгоживущего поискового процесса загрузка оплачивается один раз, для каждого нового запуска CLI — снова.

Документов	Текст, МБ	Обработка, мс	Загрузка, мс
3 376	7,11	307,87	1 040,60
6 753	14,90	647,41	1 048,26
13 507	42,77	2 131,37	1 071,02
27 014	85,42	6 023,36	1 046,85

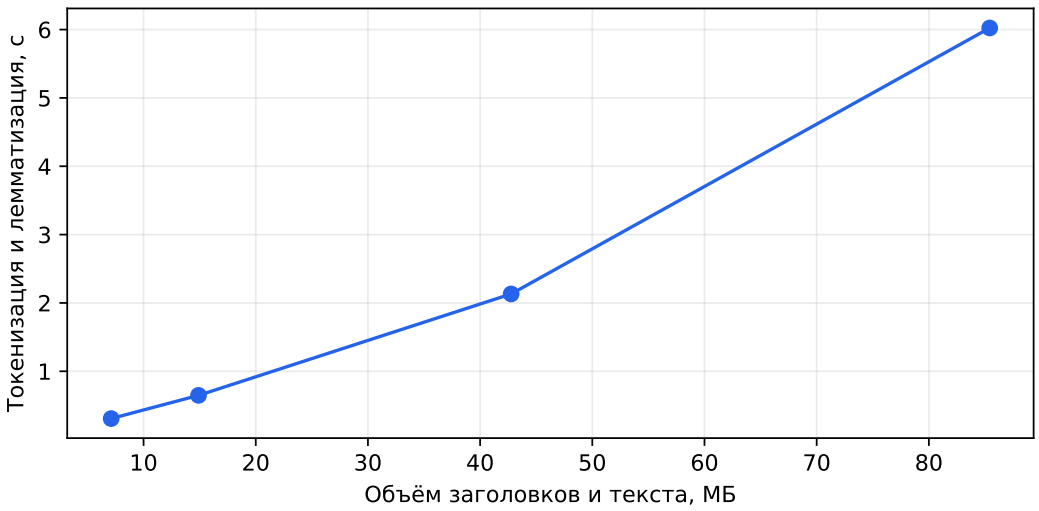


Рис. 6 — Зависимость времени обработки от объёма входного текста

Здесь КБ означает 1000 байт, МБ — 1 000 000 байт. Нельзя утверждать, что скорость оптимальна: для дальнейшей работы полезно сравнить упаковку ключей и вариантов, локальность памяти и параллельную обработку документов. Нужно учитывать и отдельную стоимость загрузки относительно короткого запроса. Повторные сырые замеры сохранены; скорость зависит от фоновой нагрузки машины.

Приведённые замеры обработки относятся к исходному снимку из 27 014 документов. Отдельный эксперимент ниже использует индекс 478 833 документов. Работа на миллионе документов данными экспериментами не проверялась.

Режим	P@5	$Recall$@5	$nDCG$@5
Точные токены	0,217	0,292	0,282
Леммы	0,383	0,794	0,824
Леммы + точное совпадение	0,433	0,844	0,862

Средние значения на фиксированном диагностическом наборе заголовков

Запрос	Токены	Леммы	С бонусом
обновления Nintendo	0,000	1,000	1,000
модификации Minecraft	0,000	0,613	0,613
модификации Elden Ring	0,000	1,000	1,000
обзоры гарнитур	0,000	1,000	1,000
mod Skyrim	0,000	1,000	1,000
reviews Witcher	0,000	1,000	1,000
games mod	0,000	0,339	0,339
release windows	0,000	1,000	1,000
Saw	0,387	0,387	0,387
Windows	1,000	0,553	1,000
S.T.A.L.K.E.R.	1,000	1,000	1,000
Windows 10	1,000	1,000	1,000

$nDCG$@5 по запросам; оценки относятся только к выбранному набору

3.3 Качество и ошибки поиска

На запросах «обновления Nintendo», «обзоры гарнитур» и «reviews Witcher» лемматизация позволяет найти заголовки с другой словоформой. При этом она не переводит русский запрос в английский: пример с Minecraft не получает все релевантные документы только за счёт морфологии.

Важный отрицательный пример — `Windows`. Без лемматизации первые пять результатов относятся к операционной системе, $nDCG@5 = 1$. После нормализации `Windows` и `windows` сближаются с `window`. В выдачу попадают публикации про *release windows*, то есть сроки выхода игр, а не ОС. Из-за равенства базовых оценок и порядка ID значение падает примерно до 0,553. Бонус за исходное точное совпадение возвращает пять заголовков про Windows и значение 1. На остальных запросах этого набора ухудшения от бонуса не наблюдалось; это не гарантия для всех запросов.

В запросе `Windows 10` дополнительный контекст уже отсекает эти ложные кандидаты. У `Saw` остаются другие трудности: название франшизы совпадает с обычным английским словом, а притяжательная форма с апострофом сохраняется токенизатором и может не совпасть с запросом. Один бонус не устраняет смысловую неоднозначность. Возможные дальнейшие улучшения — контекст, учёт названий и явные правила притяжательных форм с отдельной проверкой качества. Словарное расширение всех вариантов повышает полноту, но способно снизить точность.

Разметка, полные списки выдачи и метрики находятся в `tests/fixtures/lab4_judgments.json`, `lab4/quality.json` и локальном каталоге эксперимента. Результаты на 37 заголовках характеризуют диагностическую выборку; репрезентативной оценкой качества на всём корпусе они не являются.

3.4 Сравнение на булевом индексе большого корпуса

Дополнительный эксперимент выполнен на снимке из 478 833 документов с использованием индекса ЛР7 и поискового ядра ЛР8. Индексируются заголовок и основной текст. Словоформы и леммы хранятся в двух независимых пространствах терминов одного физического индекса; ID и метаданные документов общие. В обоих режимах применяются один разбор выражений и одинаковые операции над списками документов.

В режиме словоформ операнд после нормализации ищется непосредственно. В режиме лемм документ при индексации включается в списки всех различных разборов каждого слова. При поиске выполняется такое же преобразование операндов, и списки альтернатив объединяются. Таким образом, различие результатов определяется морфологическим представлением, а не сменой правил булевых операций или состава корпуса. Используется тот же двуязычный морфологический словарь.

Запрос	Словоформы	Леммы
Mimir AND has AND returned	0	7
татуировки AND в AND видеоиграх	8	62
игры AND doom	2667	3703
игрой AND doom	717	3703

Точное число совпадений по заголовкам и основному тексту

Запрос `Mimir AND has AND returned` не находит документов при требовании точных словоформ и возвращает семь при лемматизации. В выдаче появляется публикация о возвращении актёра, озвучивающего Мимира в God of War (рис. 2–3). Это пример полезного совпадения между различными формами слов. Слова запроса могут находиться в разных частях документа: запрос не является цитатой, порядок и соседство слов не проверяются.

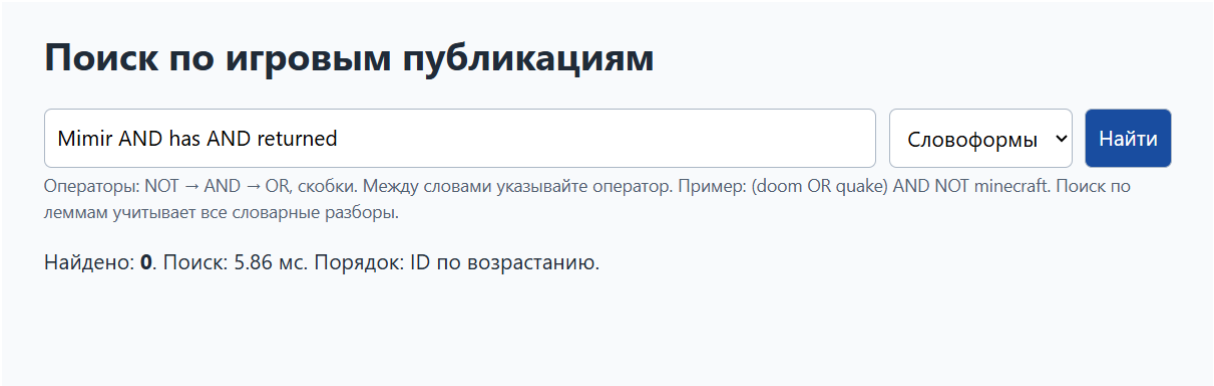


Рис. 7 — Поиск запроса Mimir AND has AND returned по словоформам

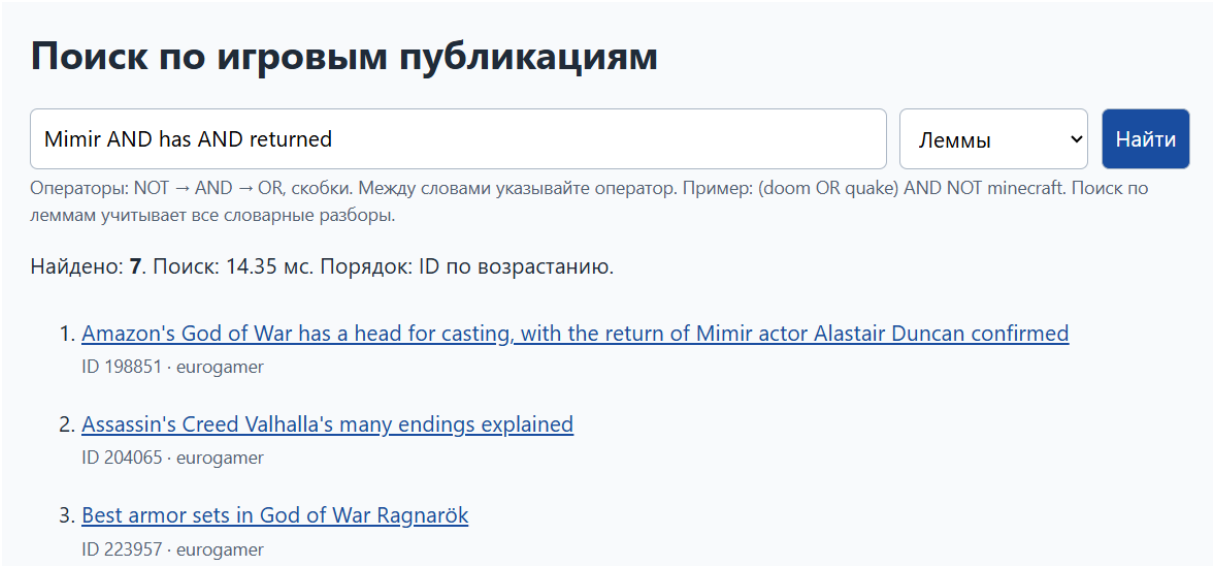


Рис. 8 — Тот же запрос в пространстве лемм

В русском примере разные формы «игры» и «игрой» приводят к одной лемме и дают одинаковое множество из 3703 документов при дополнительном условии `doom`. Поиск словоформ возвращает разные множества, поскольку требует конкретного написания. Для запроса о татуировках число кандидатов увеличивается с 8 до 62.

Увеличение числа совпадений показывает расширение множества кандидатов, но не заменяет измерения точности и полноты. Документы полного корпуса в этом эксперименте не размечены; из семи результатов запроса о Мимире не следует, что все семь релевантны. Порядок выдачи задан ID, ранжирование и бонус за точное совпадение в булевом поиске не применяются. Поэтому метрики диагностического ранжированного поиска выше не переносятся на эти списки. Скриншоты показывают отдельные запуски; время на них не является сравнительным замером производительности.

Сохранённые ответы с первыми документами, точные размеры множеств и повторные измерения находятся в `lab4/boolean_comparison/`. Запросы зафиксированы в `tests/fixtures/lab4_boolean_queries.txt`.

3.5 Повторение на пополненном корпусе

12 ноября 2026 года тот же набор булевых запросов повторён на новом снимке из 798 999 документов. Словоформы и леммы снова относятся к одним и тем же документам; ID прежнего снимка к ним не переносятся. Заголовки и текст занимают 2 445 438 731 байт, токенизатор выделил 267 962 986 токенов. Словарь остался прежним: 3 111 758 ключей. При построении нового индекса словарный разбор имели 211 669 557 токенов, изменённую форму — 91 132 122, несколько вариантов — 29 156 568. В индексе получилось 1 937 943 терма словоформ и 1 519 647 термов лемм.

Однократный повторный запуск лемматизатора без записи токенов дал 239 287 мс обработки и 249 042 мс вместе с чтением файла с диска E:. Загрузка словаря заняла 1 789 мс и входит в полное время запуска. Это не медиана пяти повторов из старой таблицы. Полный JSON сохранён в `lab4/refresh_2026-11-12.json`.

Запрос	Документы		Время, мс	
	Формы	Леммы	Формы	Леммы
Mimir AND has AND returned	0	9	8,31	31,89
татуировки AND в AND видеоиграх	14	104	20,80	41,80
игры AND doom	4522	6282	12,13	26,65
игрой AND doom	1178	6282	3,59	27,52

Число совпадений и медианное время в новом булевом индексе

Для первых двух запросов число результатов увеличилось с 0/7 до 0/9 и с 8/62 до 14/104 соответственно (словоформы/леммы). Это изменение состава корпуса, а не новая оценка точности: разметка релевантности для новых документов не выполнялась. Время — медиана трёх повторов в одном процессе каждого режима, без загрузки словаря. Повторы и первые ID сохранены в `lab4/refresh_queries_2026-11-12.json`. Старые снимки экрана выше относятся к прежнему индексу.

4 Вывод

Реализована словарная лемматизация русских и английских слов с сохранением неизвестных терминов и альтернативных разборов. Собственная хеш-таблица обеспечивает обработку текста без выделения памяти для каждого словарного токена. Модуль применён как к последовательной обработке корпуса, так и к построению и поиску по инвертированному индексу.

На небольшом размеченном наборе поиск стал находить больше нужных заголовков, но пример с Windows показал и обратную сторону: объединение словоформ иногда добавляет посторонние результаты. Сохранение исходного слова и бонус за точное совпадение улучшили результат на диагностическом наборе. На индексе 478 833 документов продемонстрировано расширение множества кандидатов и одинаковый результат для разных форм одного русского слова. На пополненном индексе из 798 999 документов те же четыре запроса сохранили это свойство; лемматизация всего снимка также измерена отдельно. Ограничения метода — отсутствие контекстного снятия омонимии, зависимость от покрытия словаря и возможное снижение точности. Дальнейшая оценка требует расширения разметки релевантности; применение бонуса за точное совпадение следует проверять на запросах с названиями и неоднозначными словами.

1 Постановка задачи

Для своего корпуса необходимо построить график распределения терминов по частотностям в логарифмической шкале, наложить на этот график закон Ципфа. Объяснить причины расхождения.

2 Метод решения

2.1 Корпус и единица подсчёта

Исследуются русскоязычные и англоязычные публикации о компьютерных играх восьми сайтов. Из MongoDB экспортирован локальный снимок из 798 999 пригодных документов: 2 445,44 МБ заголовков и основного текста в UTF-8. 8 337 публикаций участвуют только заголовком. Для всех измерений используется один и тот же снимок корпуса.

Единицей подсчёта является нормализованная словоформа, выделенная на этапе токенизации. Лемматизация не применяется: например, «игра», «игры» и «игрой» учитываются отдельно. В работе исследуется распределение словоформ. Стоп-слова сохраняются, поскольку они составляют наиболее частую часть лексики. Заголовки и основной текст учитываются совместно. Частота f равна общему числу вхождений термина, включая повторения в одном документе.

2.2 Подсчёт с ограничением памяти

Основной алгоритм написан на C++17 без контейнеров STL и готовых структур частотного словаря. Используются собственная хеш-таблица с линейным пробированием и внешняя сортировка. Для каждой группы подсчитывается число вхождений каждого термина. При заполнении выделенного буфера частоты сортируются и сохраняются частями. Последующее слияние суммирует частоты одинаковых терминов из разных частей.

Второй внешний проход упорядочивает пары по убыванию частоты внутри группы. При равенстве используется лексикографическое сравнение UTF-8. Каждому токену присваивается отдельный ранг, начиная с единицы. Весь корпус и весь итоговый словарь не требуется хранить в оперативной памяти. Объём буфера счётчика составляет 64 МиБ; дополнительная память используется для чтения текста и слияния частей.

2.3 Модели и разрезы

Пусть N — число вхождений, V — число различных терминов. Теоретическая кривая закона Ципфа имеет вид

$$f_Z(r) = \frac{C}{r}, \qquad C = \frac{N}{H_V}, \qquad H_V = \sum_{r=1}^{V} \frac{1}{r}.$$

Такая нормировка даёт сумму теоретических частот, равную N. Она не обязывает значение в первой точке совпасть с частотой самого популярного слова. В логарифмических координатах модель является прямой с наклоном -1.

Дополнительно C++ подбирает $f_P(r) = C_P / r^{\alpha}$ линейной регрессией $\ln f = \ln C_P - \alpha \ln r$ на рангах 10–10 000 в пределах словаря группы. Это отдельная оценка наклона выбранного участка распределения. При $\alpha = 1$ модель совпадает по форме с законом Ципфа. Близость показателя к единице на этом интервале не означает совпадения распределений на всех рангах.

На графике эта кривая обозначена как аппроксимация. Для каждого ранга выбранного интервала вводятся $x_r = \ln r$ и $y_r = \ln f_{\text{набл}}(r)$. По этим точкам методом наименьших квадратов оценивается прямая $y = a + bx$, то есть минимизируется

$$\sum_{r=r_{\min}}^{r_{\max}} \left(y_r - a - b x_r\right)^2.$$

После оценки коэффициентов выполняется обратное преобразование: $C_P = e^a$, $\alpha = -b$. Таким образом, по логарифму ранга оценивается логарифм частоты, а в исходных координатах получается степенная кривая. Аппроксимация описывает наклон выбранного участка данных. Для сравнения с законом Ципфа отдельно используется модель с фиксированной степенью 1.

Кроме общего распределения рассчитываются группы по сайтам и языкам источников. Язык определяется по источнику; внутри русскоязычных публикаций могут присутствовать английские названия.

В сравнительных рисунках используется относительная частота f/N каждой группы. Иначе высота линии отражала бы прежде всего объём сайта. Ранг и нормировка вычисляются внутри группы. Один документ участвует в нескольких разрезах: складывать показатели между разрезами нельзя.

2.4 Проверка корректности

План тестирования включает контрольные корпуса с заранее известными частотами, проверку ранжирования и группирования, а также сравнение результатов при разных размерах буфера. Последняя проверка подтверждает, что разбиение на части и многопроходное слияние не изменяют итоговые частоты. Подбор показателя степени проверен на распределении $f(r) = 60/r$ для рангов 1–5, для которого ожидается $\alpha = 1$. Дополнительно проверяется обработка некорректных входных данных.

Успешно пройдены 11 C++-тестов GoogleTest, одна проверка запуска программы и 5 тестов подготовки данных, сравнения и графиков. На полном корпусе проверено, что сумма частот равна числу учтённых токенов, а суммы по сайтам и языкам согласуются с общими показателями. Журнал проверок приложен в `lab5/tests.txt`.

3 Результаты

3.1 Общая статистика

Показатель	Значение
Документы	798 999
Токены N	267 962 986
Различные термины V	1 937 943
Термины с одним вхождением (hapax)	972 020
Доля hapax в словаре, %	50,16
Подобранная степень α	1,0070
Время C++ с чтением и записью, с	292,97

Измерение выполнено для Windows Release-сборки GCC 13.1.0. Приведено время одного запуска с чтением корпуса и записью частот; экспорт данных и построение графиков не учитываются. Использовался прогретый файловый кеш.

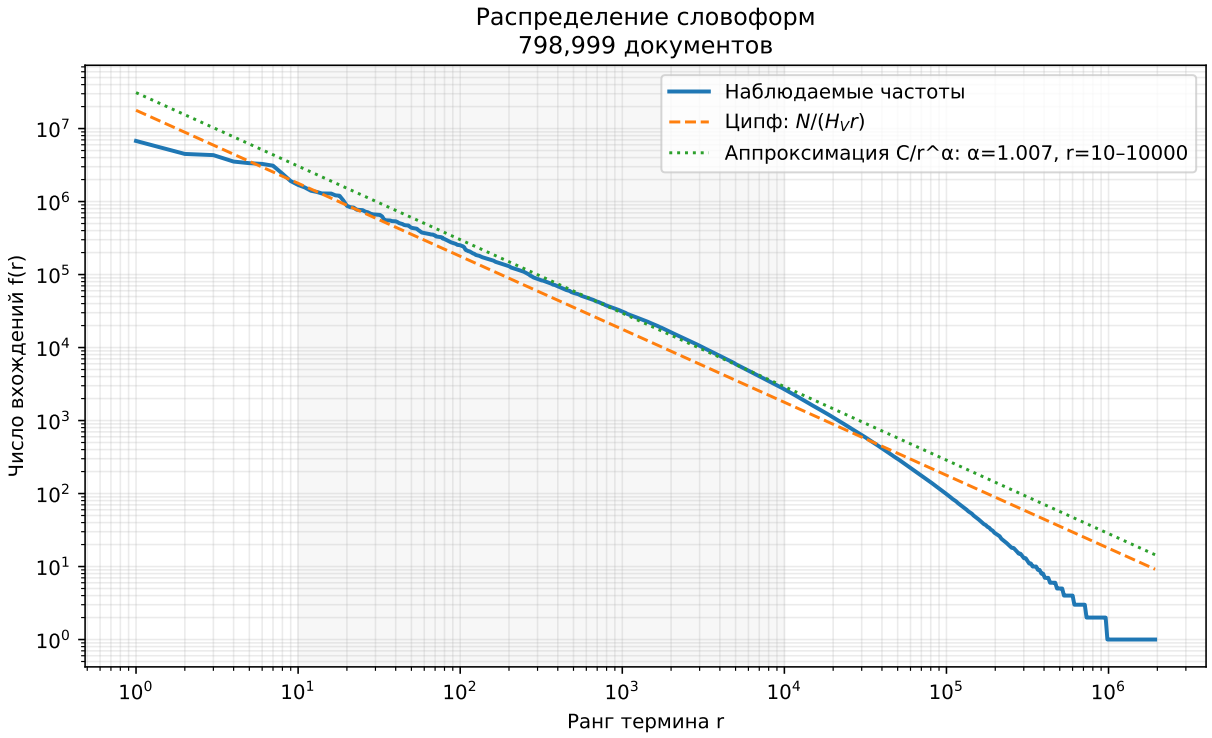


Рис. 9 — Наблюдаемые частоты, закон Ципфа и аппроксимация; серым выделен интервал оценки степени

На средних рангах распределение близко к прямой, а подобранное $\alpha = 1,0070$ близко к единице. Однако в начале кривой и в хвосте наблюдаются отклонения. Почти половина словаря встречается один раз; частоты 1, 2 и 3 образуют ступени, а не гладкую прямую. Число hapax не следует интерпретировать как число опечаток: сюда входят редкие имена, названия, словоформы и нормальные малочастотные слова.

3.2 Сравнение источников и языковых групп

Группа	Документы	Токены	Словарь	α
Весь корпус	798 999	267 962 986	1 937 943	1,007
EN-источники	242 218	126 492 332	642 033	1,191
RU-источники	556 781	141 470 654	1 426 346	0,971
Eurogamer	125 250	63 715 251	400 817	1,192
GameMAG	137 571	25 041 346	503 151	1,012
GamingOnLinux	28 211	9 821 280	110 767	1,313
Игромания	86 148	26 230 469	580 465	0,941
IXBT.games	132 459	24 079 997	431 446	1,007
PC Gamer	88 757	52 955 801	374 129	1,185
PlayGround	104 696	18 737 129	405 224	0,982
StopGame	95 907	47 381 713	851 321	0,967

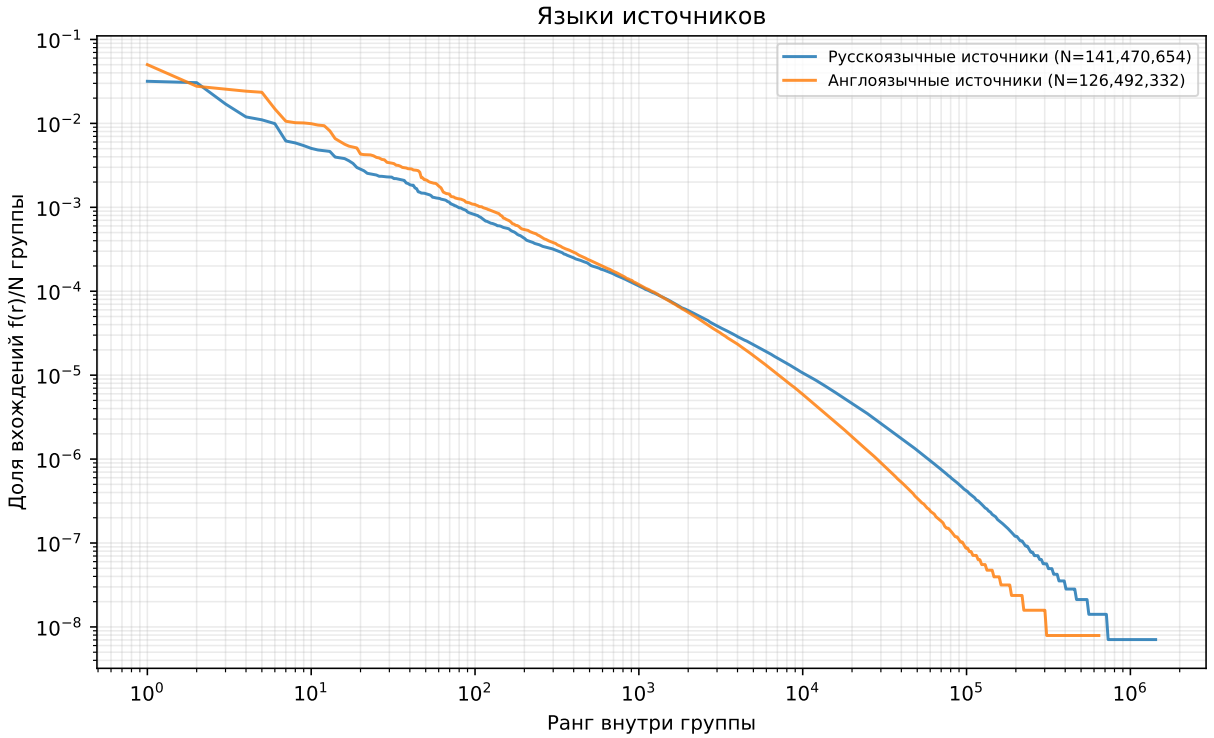


Рис. 10 — Распределения русскоязычных и англоязычных источников

У русскоязычных источников степень на выбранном интервале около 0,971, у англоязычных — 1,191. Русский словарь в этом снимке также больше. Морфологическое разнообразие русского языка может вносить вклад, но этот эксперимент не отделяет его от размера групп, состава сайтов, собственных имён и структуры публикаций. Общая смесь языков даёт свой показатель; переносить его на каждый язык отдельно нельзя.

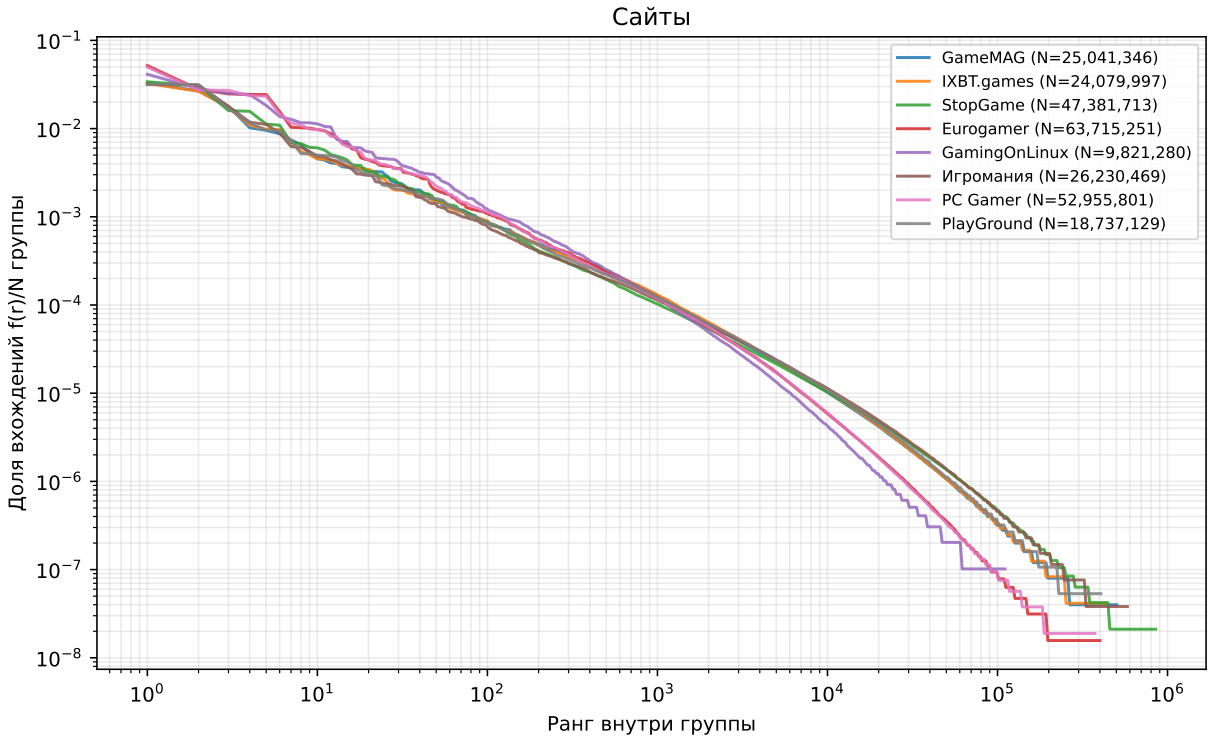


Рис. 11 — Сравнение сайтов после нормировки частот внутри источника

Причины отклонения от идеальной модели — конечный размер корпуса, ограниченная игровая тематика, смесь языков, словоформы, собственные имена и разная длина документов. Сам график не исключает влияния дублей или ошибок парсинга и не оценивает качество поисковой выдачи.

4 Вывод

В работе построены частотные распределения словоформ игрового корпуса и выполнено сравнение с законом Ципфа. На рангах 10–10 000 общий показатель $\alpha = 1,0070$ близок к единице. Основные отклонения наблюдаются среди наиболее частых терминов и в редком хвосте. Однократные термины составляют 50,16% словаря.

Сравнение групп показало различия между русскоязычными и англоязычными источниками. На результаты влияют состав корпуса, морфологическое разнообразие, собственные имена и объём выборки. Поэтому закон Ципфа описывает общую тенденцию, но не задаёт точные частоты отдельных слов.

Для информационного поиска полученное распределение показывает сочетание небольшого числа частых терминов и большого числа редких, расширяющих словарь. Это необходимо учитывать при выборе структуры словаря и способов хранения индекса. Длины списков документов зависят от документных частот и требуют отдельного подсчёта.

1 Постановка задачи

Построить булев индекс корпуса документов для последующего поиска. Индекс должен сопоставлять термину список документов, в которых он встречается, исключать повторные ID и обеспечивать доступ к спискам без последовательного просмотра исходных текстов. Структуры индекса реализовать самостоятельно на C++ с учётом обработки корпуса, не помещающегося целиком в оперативную память.

2 Метод решения

2.1 Представление данных

Используется фиксированный снимок русскоязычных и англоязычных публикаций о компьютерных играх из восьми источников. Индексируются заголовки и основной текст. Документы имеют уникальные возрастающие 64-битные ID.

Для термина t хранится список

$$L(t) = \{d : \ t \text{ встречается в документе } d\}, \qquad df(t) = |L(t)|.$$

Список упорядочен по возрастанию ID и не содержит повторений. Несколько вхождений слова в одной статье и совпадение в разных полях дают одну запись. Поэтому df отличается от частоты вхождений, рассчитанной при изучении закона Ципфа.

Индекс содержит словарь терминов, последовательные списки документов и таблицу метаданных. В записи словаря сохраняются смещение списка и число его элементов. Заголовки, URL и источники доступны по ID документа без обращения к MongoDB.

2.2 Построение с ограничением памяти

Программа написана на C++17 без готовых контейнеров STL для структур индекса. Из текста формируется поток пар «термин, ID документа». Собственная хеш-таблица исключает одинаковые пары в текущем буфере. При заполнении буфера пары сортируются собственной пирамидальной сортировкой и сохраняются в промежуточную часть.

Части сливаются с помощью собственной двоичной кучи. При совпадении пар записывается только одна из них. Если частей много, слияние выполняется в несколько проходов; за проход открыто не более 24 входов. Итоговый поток упорядочен сначала по термину, затем по ID документа. Из него последовательно записываются словарь и списки.

Размер буфера пар составляет 64 МиБ. Корпус и все списки одновременно в память не загружаются. Дополнительная память используется для текущего текстового поля, буферов слияния и морфологического словаря.

2.3 Словоформы и леммы

В одном индексе предусмотрены два независимых пространства терминов. Первое хранит исходные нормализованные словоформы. Второе строится с использованием словарного лемматизатора из ЛР4. Неизвестные слова сохраняются, а при

неоднозначном разборе документ включается в списки всех различных лемм. В каждом отдельном списке его ID остаётся уникальным.

Например, слово «стали» может добавить документ к спискам «сталь» и «стать». Это расширение множества кандидатов; контекстное снятие омонимии не выполняется. Исходное пространство позволяет позднее сравнивать точные совпадения и поиск по леммам. В отличие от частотного эксперимента здесь фиксируется наличие термина в документе, поэтому альтернативные разборы не интерпретируются как дополнительные исходные вхождения.

2.4 Чтение и проверка

Записи словаря имеют фиксированный размер и расположены в лексикографическом порядке. Поиск выполняется бинарно на диске. После нахождения записи список читается потоковым курсором. Курсоры имеют независимые позиции, что позволяет в следующей работе выполнять пересечение, объединение и разность списков.

Сохраняются версия формата, правила нормализации и контрольные суммы. Проверка готового индекса последовательно читает данные, сверяет целостность файлов, порядок словаря, уникальность ID и их принадлежность множеству документов. Незавершённый индекс не допускается к чтению.

2.5 План тестирования

GoogleTest проверяет точные списки на контрольных корпусах, устранение дублей между полями и промежуточными частями, работу с произвольными 64-битными ID, пустые документы и документы только с заголовком. Проверяются пространства словоформ и лемм, неоднозначные разборы, независимость курсоров и получение метаданных без исходного корпуса.

Синтетический корпус вызывает несколько проходов слияния. Итоговые файлы побайтово сравниваются с индексом, построенным при большем буфере. Проверяются также повреждения данных, несогласованный каталог URL, незавершённое построение и защита существующего результата.

Успешно пройдены 10 GoogleTest и одна проверка запуска CLI. Дополнительно выполнены 3 Python-теста подготовки каталога URL. Python не участвует в построении словаря или списков документов. Журнал приложен в `lab7/tests.txt`.

3 Результаты

3.1 Объём индекса

Индекс построен на 798 999 документах, содержащих 267 962 986 исходных токенов. Число токенов и размер словаря словоформ совпали с результатами ЛР5 на том же снимке.

Показатель	Словоформы	Леммы
Термины	1 937 943	1 519 647
Пары «термин, документ»	152 927 318	153 858 230

Словарь лемм меньше словаря словоформ, однако число пар «лемма, документ» несколько больше. Это связано с включением документа в списки всех различных разборов: уменьшение числа терминов не гарантирует уменьшения суммарного числа элементов списков.

Файл	Размер, МБ
terms.bin	138,30
postings.bin	2 454,28
documents.bin	51,14
strings.bin	204,92
Всего	2 848,65

ID в списках хранятся без сжатия, по 8 байт. В общий размер входят оба пространства терминов и метаданные документов. Поэтому этот размер не следует сравнивать с индексом только словоформ или со сжатым представлением. Сжатие списков и ускорение их пересечения относятся к последующим работам.

3.2 Примеры списков документов

Программа позволяет обратиться к отдельному термину без чтения корпуса. В таблице приведены реальные документные частоты и первые пять ID из сохранённых списков пространства словоформ. Многоточием обозначено продолжение списка; ID относятся к исследуемому снимку.

Термин	Пространство	*df*	Начало списка
doom	Словоформы	12 546	[69, 92, 103, 120, 226, . . .]
minecraft	Словоформы	9 852	[434, 444, 485, 509, 581, . . .]
skyrim	Словоформы	7 844	[53, 196, 374, 803, 1194, . . .]

3.3 Время и память

Показатель	Значение
Построение C++ с чтением и записью, с	2 511,47
Полная проверка готового индекса, с	170,51
Пиковый рабочий набор при построении, МиБ	209,10
Пиковый рабочий набор при проверке, МиБ	4,45
Буфер пар, МиБ	64
Морфологический словарь, МБ	166,54
Промежуточные части	293
Проходы слияния	2

Измерение выполнено для нативной Windows Release-сборки GCC 13.1.0. Приведён один прогон, а не медиана нескольких измерений. Подготовка каталога URL и

вычисление SHA-256 обвязкой не включены во время построения C++. Рабочий набор означает резидентную память процесса; размер выделенных буферов и занятые страницы могут различаться.

Результат подтверждает возможность построения индекса выбранного снимка с ограниченным буфером. Производительность на миллионном корпусе этим прогоном не проверялась. При увеличении данных дополнительно потребуются временное место на диске и проходы слияния.

3.4 Повторение работы

Инструкция сборки, запуска и инспекции отдельных списков приведена в `doc/lab7_run.md`. Реализация сохраняет индекс и предоставляет интерфейс его чтения. Разбор булевых выражений и поисковая выдача реализуются в ЛР8; оценка релевантности не относится к проверке целостности построенного индекса.

4 Вывод

Построен булев инвертированный индекс игрового корпуса. Повторные вхождения объединены в уникальные пары «термин, документ», списки упорядочены и проверены на согласованность с документным множеством. Раздельное хранение словоформ и лемм позволяет использовать одну структуру для сравнения режимов поиска.

Внешняя сортировка и слияние ограничивают объём рабочего буфера, а дисковый словарь и потоковые курсоры позволяют читать индекс без загрузки всех терминов и списков в память. Следующий этап — булев поиск по сохранённому индексу. Его ограничения на текущем этапе: отсутствие сжатия и координат слов, а также возможные лишние кандидаты при неоднозначной лемматизации.

1 Постановка задачи

Реализовать булев поиск по инвертированному индексу корпуса. Поддержать конъюнкцию, дизъюнкцию, отрицание, приоритет операций и группировку скобками. Поисковую логику и структуры данных реализовать самостоятельно на C++. Предусмотреть консольную утилиту, читающую запросы из стандартного ввода, и веб-форму с HTML-выдачей.

2 Метод решения

2.1 Множества документов и язык запросов

Используется индекс ЛР7, построенный по фиксированному снимку 478 833 публикаций об играх из восьми русскоязычных и англоязычных источников. Индексированы заголовки и основные тексты. Исходный корпус при поиске не читается; заголовки, источники и URL извлекаются из таблицы метаданных индекса.

Пусть U — множество ID всех документов, а $L(t)$ — упорядоченный список документов, содержащих термин t. Тогда

$$R(A \text{ AND } B) = R(A) \cap R(B), \qquad R(A \text{ OR } B) = R(A) \cup R(B),$$

$$R(\text{NOT } A) = U \setminus R(A).$$

Множество U включает документы без токенов. ID являются 64-битными числами и могут иметь пропуски: дополнение строится по таблице документов, а не по числовому диапазону ID.

Приоритет операций: `NOT`, затем `AND`, затем `OR`. Скобки меняют порядок вычислений. Допускаются символьные аналоги `!`, `&`, `|`. Между словами требуется явная операция. Например, `doom OR quake AND NOT minecraft` означает `doom OR (quake AND (NOT minecraft))`.

2.2 Разбор и потоковое вычисление

Рекурсивный спуск строит собственное дерево выражения. Каждый лист обращается к дисковому словарю и открывает независимый курсор списка документов. Внутренний узел хранит текущие ID дочерних потоков и последовательно выполняет операцию над ними.

При пересечении продвигается поток с меньшим ID; совпавший ID выдаётся один раз. При объединении выдаётся меньший ID, а совпадение продвигает оба потока. Разность пропускает ID левого потока, присутствующие в правом. Поэтому порядок и уникальность сохраняются на каждом уровне дерева. Промежуточные множества и полная выдача в памяти не сохраняются.

Операция над двумя отсортированными списками длины p и q требует $O(p+q)$ шагов в худшем случае. Память определяется деревом, текущими элементами потоков, файловыми буферами и словарём лемматизации, а не общим числом найденных документов. Запрос ограничен 8192 байтами, 64 операндами и 64 уровнями вложенности; после раскрытия лемм допускается не более 1024 узлов. Превышение возвращает явную ошибку.

2.3 Словоформы и леммы

Операнды нормализуются тем же токенизатором, что и документы. Режим словоформ обращается к исходному пространству терминов. В режиме лемм каждый операнд разбирается словарём из ЛР4, а списки всех его различных лемм объединяются. Таким образом, для множества разборов $M(w)$

$$L_{\text{query}}(w) = \bigcup_{\ell \in M(w)} L_{\text{lemma}}(\ell).$$

Это соответствует политике построения индекса: документ включён в список каждой допустимой леммы его слов. Неизвестные слова сохраняются. Например, отрицание `NOT стали` исключает документы из объединения списков «сталь» и «стать». Контекстное снятие неоднозначности отсутствует. Совпадение словаря запроса со словарём индекса проверяется по сохранённому 64-битному отпечатку файла.

2.4 Интерфейсы

Консольная утилита принимает по одному UTF-8-запросу в строке стандартного ввода и выдаёт по одному JSON-объекту в строке стандартного вывода. Для каждого запроса возвращаются точное число совпадений, время вычисления и ограниченный список документов с ID, заголовками, URL и источниками. Ошибка одной строки не препятствует обработке следующих.

Веб-форма предоставляет ввод выражения и переключение режима. Выдача содержит число совпадений и первые 20 заголовков со ссылками. Python передаёт запрос C++-процессу через UTF-8 stdin и преобразует полученные метаданные в HTML; поисковых структур в Python нет. Для каждого обращения формы запускается отдельный процесс, поэтому время HTTP-ответа дополнительно включает загрузку словаря.

Оба интерфейса упорядочивают результаты по ID, без ранжирования. Ограничение размера отображаемой выдачи не прекращает вычисление: для точного числа совпадений поток ID читается до конца.

3 План и результаты тестирования

На контрольном корпусе проверяются приоритеты, скобки, повторные операнды, тождества множеств, законы де Моргана и двойное отрицание. Для 64 автоматически составленных выражений в каждом из двух режимов результаты сравниваются с независимой таблицей истинности для всех сочетаний трёх терминов. Используются термины, одинаково представленные в обоих пространствах; преобразование словоформ в леммы проверяется отдельно. В корпус включены пустой документ и произвольные 64-битные ID.

Отдельно проверяются нормализация, оба пространства терминов, неоднозначные леммы, неизвестные слова и несовпадающий словарь. Проверки CLI охватывают точное число совпадений при ограниченной выдаче, корректность JSON, экранирование метаданных, UTF-8, CRLF, последнюю строку без перевода строки и продолжение после ошибки. Некорректные выражения и превышения ограничений отклоняются.

Успешно пройдены 11 GoogleTest и одна процессная проверка CLI. Три Python-теста проверяют передачу UTF-8 в нативную программу, HTTP-форму, обработку ошибок и экранирование HTML. Журнал с отдельными тест-кейсами приложен в `lab8/tests.txt`. Команды пересборки и прямого запуска GoogleTest приведены в `doc/lab8_run.md`.

4 Результаты на корпусе

4.1 Число совпадений и время

В таблице приведены точные размеры множеств и медианы 3 повторений каждого запроса в одном процессе для каждого режима. Измерено время нормализации, разбора выражения и полного получения потока ID. Загрузка словаря, чтение метаданных и вывод JSON в это время не включены. Применена нативная Windows Release-сборка GCC 13.1.0; файловый кеш между повторениями не очищался.

	Документы		Время, мс	
Запрос	Формы	Леммы	Формы	Леммы
doom	7469	8112	0,86	0,61
doom AND quake	975	989	1,31	1,03
doom OR quake	9597	10271	1,4	1,11
(doom OR quake) AND NOT minecraft	9291	9938	1241,91	979,29
NOT doom	471364	470721	1261,57	1030,08
games AND doom	2697	4653	10,86	12,48
игры AND doom	2667	3703	13,63	12,76
игрой AND doom	717	3703	3,95	10,99
(skyrim OR fallout) AND bethesda	4425	4427	2,56	2,0
doom AND zzzmissingterm12345	0	0	0,95	0,34

Для `doom` исходное число совпадений 7469 совпадает с документной частотой из ЛР7. Пересечение с `quake` уменьшает множество до 975 документов, объединение увеличивает его до 9597. Отрицание `NOT doom` возвращает $478833 - 7469 = 471364$ документа. Пересечение с отсутствующим термином пусто в обоих режимах.

Запросы «игры AND doom» и «игрой AND doom» дают различные множества словоформ: 2667 и 717 документов. После лемматизации обе формы приводят к одному результату из 3703 документов. Это демонстрирует устранение различий между падежными формами. Увеличение количества кандидатов само по себе не доказывает рост точности или полноты относительно информационной потребности: для такой оценки необходима отдельная разметка релевантности.

4.2 Память и ограничения производительности

Пиковый рабочий набор поискового процесса составил 4,59 МиБ для словоформ и 163,47 МиБ для лемм. Разница определяется главным образом загруженным морфологическим словарём. Индекс размером около 1,72 ГБ целиком в память не загружался. Измерен рабочий набор процесса, без системного файлового кеша.

Отрицание перечисляет множество документов, поэтому запросы с `NOT` существенно медленнее простых объединений и пересечений. В текущем дереве выраже-

ние `A AND NOT B` вычисляет поток $U \setminus B$ и пересекает его с A. Преобразование в непосредственную разность $A \setminus B$ позволило бы избежать полного перечисления U. Планирование по документным частотам и прыжки по спискам также могут уменьшить число чтений; в данной работе они не применяются.

4.3 Пример веб-поиска

На рис. 1 показано выполнение запроса «покупка AND студии AND bungie» в пространстве словоформ. Найдены 22 документа. Условие требует наличия всех трёх слов в индексируемых полях документа; их соседство и порядок не проверяются. Поэтому совпадение может находиться в основном тексте и не быть полностью представлено в заголовке выдачи.

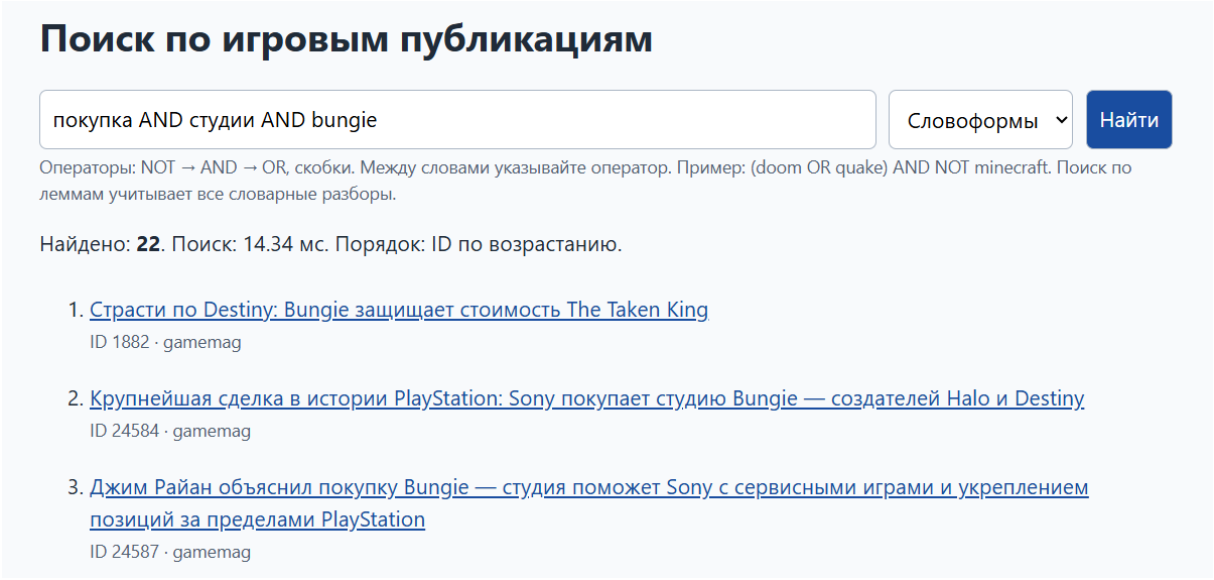


Рис. 12 — Булев запрос с тремя условиями AND в веб-интерфейсе

4.4 Повторное измерение на расширенном корпусе

После пополнения корпуса построен отдельный снимок из 798 999 документов вместо 478 833. Новый индекс прошёл полную проверку; исходное число токенов совпало с независимым подсчётом частот. На нём повторён тот же набор булевых запросов в двух пространствах терминов. Результаты старого опыта и снимок интерфейса выше относятся к прежнему индексу.

Запрос	Документы		Время, мс	
	Формы	Леммы	Формы	Леммы
doom	12546	13618	1,03	1,29
doom AND quake	1666	1689	1,23	1,88
doom OR quake	16032	17158	1,58	2,14
(doom OR quake) AND NOT minecraft	15488	16576	2,82	2,69
NOT doom	786453	785381	2809,16	3114,47
games AND doom	4561	7850	11,94	14,03
игры AND doom	4522	6282	15,15	16,31
игрой AND doom	1178	6282	3,33	20,4
(skyrim OR fallout) AND bethesda	7626	7628	3,05	4,18
doom AND zzzmissingterm12345	0	0	0,61	0,8

На новом индексе `doom` встречается в 12 546 документах по словоформам и в 13 618 по леммам. `NOT doom` возвращает $798999 - 12546 = 786453$ документа. Времена являются медианами трёх повторений с сохранённым системным файловым кешем.

5 Вывод

Реализованы разбор булевых выражений и потоковое вычисление точных множеств документов по сохранённому индексу. Поддержаны словоформы, леммы и согласованное раскрытие морфологической неоднозначности. Консольная утилита и веб-форма используют одно поисковое ядро C++.

Проверки подтверждают соответствие выдачи булевой алгебре, в том числе для пустых документов, неизвестных терминов и произвольных ID. Поиск испытан на снимках из 478 833 и 798 999 документов; работа на миллионном корпусе этими измерениями не проверялась. Основные ограничения — полное перечисление корпуса при отрицании, отсутствие ранжирования, координат слов и контекстного снятия омонимии. Дальнейшие направления: планирование выражений, прыжки по спискам, цитатный поиск и оценка релевантности на размеченных запросах.

1 Постановка задачи

Дополнить поисковую систему координатами слов и цитатным поиском. Запрос в кавычках должен находить документы, содержащие последовательность слов в заданном порядке на соседних позициях. Обеспечить сочетание фраз с булевыми операторами, консольный ввод-вывод и веб-интерфейс. Основные алгоритмы и структуры данных реализовать самостоятельно на C++.

2 Метод решения

2.1 Зачем нужны координаты

Булев индекс хранит факт присутствия слова в документе. Поэтому запрос `doom AND eternal` удовлетворяется даже тогда, когда слова находятся в разных предложениях или в обратном порядке. Список документов не содержит информации, необходимой для проверки фразы.

Координатное расширение сохраняет для каждого термина тройки (d, f, p): документ, поле и позиция токена внутри поля. Например, текст «doom new eternal» содержит координаты $p(\texttt{doom}) = 0$ и $p(\texttt{eternal}) = 2$. Он соответствует конъюнкции, но не фразе `"doom eternal"`, поскольку разность позиций равна двум вместо единицы.

Для фразы из m токенов $w_0, \ldots, w_{m-1}$ требуется

$$\exists (d, f, p): \quad (d, f, p + i) \in C(w_i), \qquad 0 \le i < m,$$

где $C(w)$ — множество координат слова. Иными словами, нужно найти общую начальную позицию в одном документе и поле.

2.2 Корпус и представление данных

Используется снимок корпуса ЛР7–ЛР8 из 798 999 документов. Индексируются заголовок и основной текст. Позиции нумеруются от нуля отдельно в каждом поле; совпадение через границу полей исключается. Слова нормализуются тем же токенизатором, что и в предыдущих работах. Регистр и пунктуация не являются самостоятельными условиями совпадения. Таким образом, цитатный поиск проверяет последовательность нормализованных токенов, а не побайтовое равенство исходной строки.

К существующему булевому индексу добавлены два файла. Первый сопоставляет номеру термина смещение и длину списка координат, по 16 байт на запись. Второй хранит сами координаты, по 8 байт на вхождение: 32-битный порядковый номер документа и 32-битное представление поля и позиции. Старший бит второго числа задаёт поле, остальные 31 бит — позицию токена.

Порядковый номер документа переводится в исходный 64-битный ID через таблицу базового индекса. Это сохраняет поддержку ID с пропусками и значений больше 2^{32}. Формат координат ограничен числом документов не более $2^{32} - 1$, числом терминов не более $2^{32} - 2$ и позициями меньше 2^{31} в одном поле. Превышения явно отклоняются.

Заголовки файлов содержат версию формата, размеры, контрольные суммы, отпечатки корпуса, словаря и заголовков базового индекса. При построении исход-

ный поток проверяется по ID, числу документов и полной контрольной сумме. Незавершённые файлы и расширение, относящееся к другому булевому индексу, не допускаются к поиску.

2.3 Построение с ограниченным буфером

Собственная хеш-таблица переводит строковые термины в номера словаря. Из корпуса последовательно формируются тройки «термин, порядковый номер документа, поле/позиция». По мере заполнения буфера они сортируются собственной стабильной поразрядной сортировкой и записываются во временные части. Два массива сортировки вместе занимают не более 64 МиБ. Дополнительная память используется для словарей и текущего поля текста.

Отсортированные части сливаются собственной минимальной кучей. Одновременно открыто не более 24 входов, поэтому при большом количестве частей выполняется несколько проходов. Повторяющиеся координаты удаляются. После завершения промежуточные файлы удаляются, а индекс получает признак готовности. При записи контролируется резерв свободного места 4,5 ГиБ. Корпус и булев индекс остаются неизменными.

2.4 Потоковое вычисление фраз

Для слова с номером i координаты сдвигаются на $-i$:

$$C'_i = \{(d, f, p - i) : (d, f, p) \in C(w_i),\ p \geq i\}.$$

Пересечение $C'_0 \cap \ldots \cap C'_{m-1}$ даёт начала совпадающих фраз. Потоки упорядочены по документу, полю и позиции. Если текущие ключи не совпадают, продвигаются потоки с меньшим ключом. При совпадении документ выдаётся один раз независимо от числа вхождений фразы. Полные списки и промежуточные результаты в память не загружаются.

Проверка учитывает повторяющиеся слова: фраза «red red» требует двух соседних позиций, а не одного наличия термина. В худшем случае для m слов и списков длины n_i затраты текущей реализации составляют $O(m \sum_i n_i)$ сравнений ключей. Дополнительный перебор возможен при объединении альтернативных лемм; число слов и альтернатив ограничено интерфейсом.

2.5 Два пространства и интерфейсы

Словоформы и леммы имеют независимые списки координат, но общую таблицу документов. В режиме лемм варианты одного слова объединяются до сдвига координат. Все разборы сохраняют исходную позицию: «стали» может соответствовать «сталь» и «стать», но не создаёт искусственную последовательность из двух слов. Фраза по леммам допускает разные грамматические формы, сохраняя требование к порядку и соседству.

Фраза становится листом дерева булевого запроса ЛР8. Например, `"doom eternal" AND NOT quake` требует фразу и исключает документы со словом `quake`. Консольная утилита получает по одному UTF-8-запросу в строке stdin и выдаёт JSONL в stdout. Веб-форма передаёт запрос тому же ядру C++ и показывает заголовки со ссылками. Python обслуживает интерфейс и измерения, не вычисляя поисковые множества.

Сохраняются точный подсчёт совпадений и порядок выдачи по ID. Для фразы допускается до 64 токенов и 512 координатных потоков. Синтаксические ошибки, отсутствие координат и превышения ограничений возвращаются явно. Обычные булевы запросы работают и без расширения.

3 План и результаты тестирования

GoogleTest проверяет соседство, прямой и обратный порядок, границы полей, повторяющиеся слова, несколько вхождений и неизвестные термины. Проверяются фразы в обоих пространствах и их сочетание с AND, OR и NOT. Для 14 фраз длиной от одного до трёх слов в каждом режиме полный список ID сравнивается с независимой проверкой скользящим окном на восьми текстах.

Словарные примеры проверяют разные русские словоформы и сохранение неоднозначных лемм на одной позиции. Проверки CLI охватывают JSON, точное число совпадений при ограниченной выдаче и продолжение после ошибочной строки. Проверяются пустой индекс, произвольные 64-битные ID, непарные кавычки и пустые фразы.

Отдельный корпус создаёт более 24 временных частей и два прохода слияния. Результат побайтово совпадает с индексом, построенным при большем буфере. Проверяются усечение и повреждение файлов, незавершённое построение, чужой корпус и словарь, защита существующего результата и дисковый резерв.

Успешно выполнены 12 GoogleTest и одна процессная проверка CLI. Два Python-теста проверяют передачу кавычек и пути координат в нативную программу, а также отображение ошибки поиска. Журнал приложен в `lab10/tests.txt`.

4 Результаты на корпусе

4.1 Объём и построение

На 267 962 986 исходных токенах сохранено 567 184 357 координат в двух пространствах. Число координат включает альтернативные леммы, поэтому не равно числу исходных вхождений слов. Координатное расширение занимает 4592,80 МБ дополнительно к индексу ЛР7.

Показатель	Значение
Документы	798 999
Время построения C++, с	1767,47
Полная проверка расширения, с	252,08
Пиковый рабочий набор при построении, МиБ	362,99
Координаты и таблица смещений, МБ	4592,80

Измерение выполнено в нативной Windows Release-сборке GCC 13.1.0. Время построения включает предварительную полную проверку булева индекса, чтение корпуса, морфологию, сортировку и запись. Приведён один прогон построения; память означает рабочий набор процесса и не включает системный файловый кеш.

Пример начала координатного списка слова `doom`:

```
occurrences 33327
document_id field position
69 body 39
69 body 44
92 title 0
92 body 0
92 body 4
92 body 24
92 body 152
92 body 432
92 body 971
92 body 1557
```

Каждая строка задаёт ID, поле и номер токена. Одному документу могут соответствовать несколько строк; это отличает координатный список от булева списка уникальных ID.

4.2 Сравнение конъюнкции и фразы

Таблица содержит точные количества совпадений и медианы трёх повторений в одном процессе каждого режима. Время C++ включает нормализацию, разбор запроса и получение всех ID, но исключает загрузку словаря и вывод метаданных. Файловый кеш между повторениями не очищался.

Запрос	Документы		Время, мс	
	Формы	Леммы	Формы	Леммы
doom AND eternal	2401	2435	1,32	1,31
"doom eternal"	2210	2210	11,20	12,96
"eternal doom"	33	33	2,68	2,55
"doom eternal" AND NOT quake	2067	2066	12,02	14,62
god AND of AND war	11253	12298	17,92	19,08
"god of war"	10062	10071	176,87	188,94
"дата выхода"	9612	21618	43,05	90,54
"даты выхода"	4921	21618	26,57	92,12

Для одной и той же последовательности слов цитатная выдача является подмножеством конъюнкции: добавлены ограничения на координаты. Сравнение обратной последовательности показывает, что порядок является частью условия. Русские примеры позволяют сопоставить точные словоформы и грамматически нормализованную фразу. Количество совпадений не заменяет оценку релевантности: разметка полного корпуса для этих запросов не выполнялась.

Иллюстрация требования к порядку приведена на рис. 1 и 2. По словоформам фраза «god of war» встречается в 5804 документах, в том числе в заголовках публикаций об одноимённой игре. При перестановке тех же слов в «war of god» результат пуст. Скриншоты показывают отдельные запуски веб-интерфейса; показанное на них время не является медианой из таблицы.



Рис. 13 — Поиск фразы «god of war» по словоформам: 5804 документа

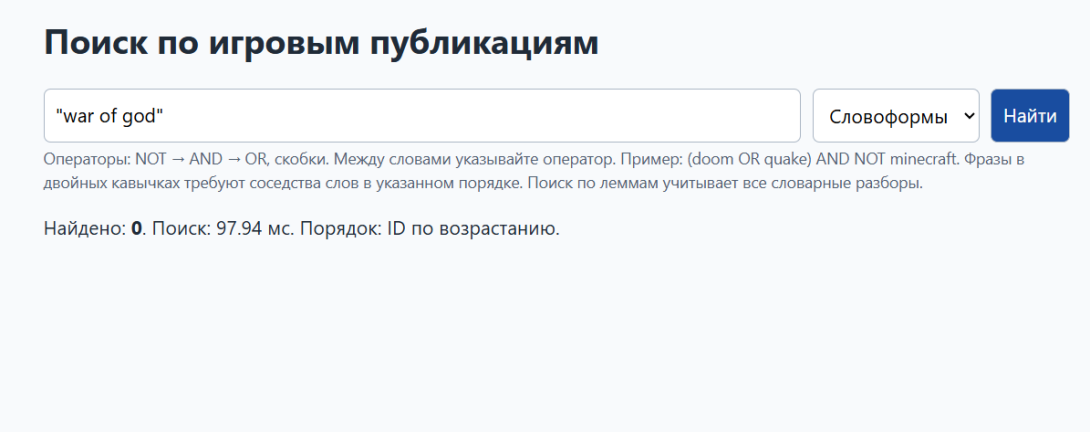


Рис. 14 — Обратный порядок слов «war of god»: совпадений нет

4.3 Использование цитат в ранжированном поиске

Координатный поиск фразы теперь применяется и как обязательный фильтр перед выдачей TF-IDF. Для нового снимка из 798 999 документов запрос `"doom eternal"` по словоформам даёт 2 210 документов как в булевом, так и в ранжированном режиме. Без кавычек TF-IDF допускает разрозненные слова и возвращает 15 334 документа. При цитатном запросе порядок документов по TF-IDF меняется, но условие полного совпадения фразы сохраняется. Данные проверки находятся в `lab13/phrase_filter_2026-11-12.json`.

5 Вывод

Реализовано координатное расширение инвертированного индекса и поиск последовательностей соседних токенов. Общая модель булевого поиска сохранена: фразы можно соединять логическими операциями и использовать в отрицании. Раздельная нумерация полей исключает ложные совпадения через границу заголовка и текста.

Цитатный поиск позволяет выразить требование к устойчивой фразе или названию, которое невозможно проверить по одному наличию слов. В режиме лемм требование к координатам сочетается с морфологическим расширением. Ограничения реализации — отсутствие сжатия, прыжков, ранжирования и контекстного снятия омонимии; пунктуация и границы предложений отдельно не проверяются. Работа на миллионном корпусе этими измерениями не подтверждается. Следующие направления развития — сжатие координат и ускорение пересечения длинных списков.

1 Постановка задачи

Разработать способ сжатия инвертированного индекса без потери информации. Обеспечить чтение сжатых списков при выполнении поисковых запросов и проверить совпадение выдачи с исходным индексом. Измерить размер данных, время поиска, чтения и декодирования списков. Подготовить консольную демонстрацию и автотесты. Основные алгоритмы и структуры данных реализовать на C++.

2 Метод решения

2.1 Состав индекса и область сжатия

В инвертированном индексе различаются словарь терминов, списки документов (постинги), координаты вхождений и сведения о самих документах. Исходный корпус текстов и таблица документов относятся к другим данным. В данной работе сжаты словарь терминов, постинги и координаты. Таблица документов не менялась. Морфологический словарь лемматизатора хранится отдельно и не входит в измеряемый индекс. Экономия оценивается для каждой структуры и для всего рабочего индекса.

Булев индекс хранит для каждого термина возрастающие идентификаторы документов в виде 64-битных целых. Координатное расширение хранит пары из порядкового номера документа и номера токена с признаком поля; каждая пара занимает 8 байт. Списки упорядочены, поэтому соседние значения часто различаются на небольшую величину.

2.2 Кодирование разностей и чисел

Для списка $d_1 < d_2 < \cdots < d_n$ записывается $\Delta_i = d_i - d_{i-1}$, где $d_0 = 0$. Например, список $(3, 8, 42)$ превращается в $(3, 5, 34)$. Для координаты (o, f, p) отдельно кодируются $o - o_{\text{prev}}$ и $2\Delta p + f$, где $f \in \{0, 1\}$ обозначает заголовок или основной текст. При смене документа либо поля $\Delta p = p$, иначе $\Delta p = p - p_{\text{prev}}$. Так сохраняется строгий порядок координат и не смешиваются позиции разных полей.

Неотрицательные числа записываются кодом Variable Byte (VB) в варианте лекции. В каждом байте семь бит данных; младшие группы идут первыми. Старший бит равен нулю у промежуточных байтов и единице у последнего. Поэтому 128 записывается двумя байтами `00 81`. Значения до 127 занимают один байт, до 16 383 — два. Декодер отклоняет усечённые, избыточные и переполненные представления.

2.3 Блочное сжатие словаря

Термины упорядочены лексикографически внутри двух пространств. Они разбиты на блоки по 16 записей; для каждого блока хранится одно смещение. Первый термин записывается полностью, каждый следующий — длиной общего префикса с предыдущим и собственным суффиксом. Например, для `game` и `games` вторая строка представляется длиной префикса 4 и суффиксом `s`. В блоке также закодированы пространство термина, число документов и начальное смещение постингов. Поиск бинарно находит блок по первому термину и последовательно просматривает до 16 записей. Полный словарь в память не загружается.

2.4 Формат и чтение

Для каждого термина сохраняются смещение и длина его сжатого списка. Это позволяет открыть нужный поток без загрузки всех списков в память. Сжатое расширение содержит четыре файла числовых потоков и две структуры словаря: таблицу смещений блоков и поток префиксных записей. Заголовки имеют номер формата, размеры, контрольные суммы FNV-64 и отпечаток исходного индекса. Незавершённая сборка помечается отдельно. Сжатие не меняет токенизацию, лемматизацию, язык запросов или порядок выдачи.

Для проверки физической экономии подготовлена отдельная рабочая копия индекса. В ней сохраняются таблица документов и таблица терминов координатного расширения. От исходных таблицы терминов и двух больших числовых потоков остаются 80-байтные заголовки для привязки форматов. Из общего файла строк удаляются строки терминов, а строки заголовков, URL и источников сохраняются. Поиск в рабочей копии не требует исходных строк терминов, массивов постингов и координат. Исходный индекс хранится отдельно как эталон сравнения.

3 Журнал выполнения

Сначала построены потоки разностей по проверенным исходным индексам. Из-за произвольных 64-битных ID документов нельзя кодировать усечённые 32-битные идентификаторы; используются полные 64-битные разности. Координаты сжимаются отдельно от списков документов, поскольку цитатный поиск должен восстанавливать порядок и соседство слов без потери информации. При проверке самостоятельной копии обнаружилось, что маркер незавершённой сборки находился в её корне, а поиск проверял только вложенные каталоги. Проверка корневого маркера добавлена в читатель сжатого словаря и закреплена отдельным тестом: незавершённая копия отклоняется, после снятия маркера готовая копия снова открывается.

Для разбора замедления фразового поиска добавлен диагностический режим. Один и тот же VB-декодер работает с файлом и с уже загруженным массивом байтов, что позволяет отдельно измерить чтение и восстановление значений.

4 План и результаты тестирования

План тестирования включает:

1. проверки граничных значений переменного кода, избыточных форм и усечения;
2. построение на малом корпусе, сравнение каждого термина, ID и координаты;
3. сравнение булевых и фразовых запросов в пространствах словоформ и лемм, включая повторные слова, отрицание и 64-битные ID;
4. отказ при повреждении, неполной сборке, чужом индексе, повторной записи в существующий каталог и недостатке свободного места;
5. проверку полного корпуса и повторные измерения запросов.
6. сравнение первых результатов и P@5 на сохранённой разметке текстов для исходного и сжатого индекса.

Нативные проверки реализованы в GoogleTest; запуск и данные измерений приложены к исходникам. Программа `compact_index verify-packed` сверяет контрольные суммы, границы списков и *каждый* восстановленный термин, ID и координату с исходными файлами. Пройдены все девять проверок группы ЛР11 и 123 проверки общего набора CTest.

5 Результаты на корпусе

На корпусе из 478 833 документов построен индекс с 2 545 616 терминами в двух пространствах. Исходные списки документов и координаты занимают 4140,27 МБ. Четыре сжатых файла вместе с таблицами смещений занимают 1210,54 МБ: экономия 70,8 %. Полное сравнение декодирования выполнено за 86,0 с. Полный исходный индекс, включая словарь и метаданные документов, занимает 4443,34 МБ. Отдельная рабочая копия со сжатыми структурами занимает 1400,56 МБ; фактическая экономия для всего индекса составляет 68,5 %.

Структура	Исходный размер, МБ	Сжатый размер, МБ
Списки документов	1 455,88	224,28
Координаты	2 684,39	904,80
Таблицы смещений	—	81,46
Словарь терминов	138,83	25,77

Словарь терминов уменьшился на 81,4 % благодаря хранению общих префиксов, переменному кодированию метаданных и одному смещению на блок из 16 терминов. Здесь и далее МБ означает 10^6 байт. Исходные постинги хранят произвольные 64-битные ID документов, поэтому процент экономии относится именно к этому формату. Прямое сравнение с лекционными замерами для 32-битных ID было бы некорректным.

Для набора булевых и фразовых запросов в обоих пространствах совпали полное число найденных документов и показанные ID. Полная проверка списков отдельно подтвердила точное восстановление данных. Порядок выдачи остался по возрастанию ID.

5.1 Сохранение качества выдачи

Для трёх запросов из ЛР13 просмотрены заголовки и тексты первых пяти документов булева AND/ID по исходному снимку MAIIR3 из 478 833 публикаций. Метка 1 означает, что игра является основной темой публикации, 0 — что она упомянута попутно. По этим меткам рассчитана доля релевантных документов среди первых пяти, P@5. Во всех вариантах сверены число ответов, ID и заголовки первой пятёрки.

Запрос	Исходный	Сжатые списки	Рабочая копия
doom eternal	0,40	0,40	0,40
god of war	0,80	0,80	0,80
cyberpunk 2077	0,60	0,60	0,60
Среднее	0,60	0,60	0,60

На каждом запросе первые пять документов и полное число совпадений оказались одинаковыми, поэтому P@5 сохранилась точно; падение составило 0. Разметка 15 документов предварительная и выполнена помощником, а не независимым экспертом. Критерии и обоснования сохранены в `../tests/fixtures/lab11_article_judgments.json`. При сжатии без потерь роста релевантности не ожидается: алгоритм поиска и порядок ID не меняются. Три запроса и 15 размеченных документов не позволяют оценить качество всей системы. Главный аргумент о сохранении качества при сжатии — точное совпадение восстановленных списков и выдачи; P@5 служит иллюстрацией на понятных запросах. Исходные ответы и расчёт сохранены в `lab11/quality.json`.

5.2 Время поиска

Для дополнительного эксперимента выбраны фразы и булев запрос из тех же слов. В таблице приведены медианы 7 повторений после одного прогревочного пакета запросов. Сжатый режим использует самостоятельную рабочую копию. Время включает разбор запроса и перечисление всех совпадений, но исключает открытие индекса, загрузку морфологического словаря и вывод метаданных. Кэш ОС не очищался, поэтому это не измерение холодного диска.

Пространство	Запрос	Исходный, мс	VB, мс
словоформы	`"god of war"`	127,19	166,87
словоформы	`god AND of AND war`	14,49	10,47
словоформы	`"doom eternal"`	5,57	6,38
словоформы	`"eternal doom"`	1,70	2,55
леммы	`"god of war"`	127,86	166,31
леммы	`god AND of AND war`	16,03	11,33
леммы	`"doom eternal"`	6,54	5,89
леммы	`"eternal doom"`	1,96	2,71

Запросы в кавычках выполнялись как фразы, а строки с AND — как пересечение списков документов без проверки соседства слов. Фраза «god of war» по словоформам выполнялась за 127,19 мс на исходном индексе и за 166,87 мс на сжатом. Сжатие уменьшило размер данных, но в этом запросе время выросло на 31,2 %.

5.3 Чтение и декодирование

Отдельно измерены полные списки терминов `god`, `of` и `war`. Чтение — это `seek` и `fread` в заранее выделенный и заполненный буфер. Декодирование выполняется из памяти без обращений к файлу: для исходного формата читаются фиксированные целые, для VB восстанавливаются разности и поля координат. Оба варианта вычисляют одинаковую контрольную сумму восстановленных значений. Результаты сравниваются после замера; расхождений не обнаружено. Порядок исходного и сжатого вариантов чередуется между повторениями.

В таблице показаны координаты в пространстве словоформ, время дано в мс. Для каждого списка выполнен один предварительный прогрев; приведены медианы 7 повторений.

Термин	Чтение		Декодирование	
	Исходный	VB	Исходный	VB
`god`	0,01	0,01	0,02	0,22
`of`	5,32	1,26	2,30	18,50
`war`	0,04	0,02	0,07	0,77

Для связи с реальным поиском также измерен полный проход штатного курсора: открытие потока, чтение и восстановление всех координат, вычисление той же контрольной суммы и закрытие потока.

Термин	Исходный курсор, мс	VB-курсор, мс
`god`	0,75	1,24
`of`	82,93	119,72
`war`	2,32	4,61

Особенно заметна разница на `of`: список содержит 1 997 146 координат и занимает 15,98 МБ в исходном формате против 4,37 МБ в VB. Сжатые байты читаются быстрее, но их восстановление требует больше времени. Штатный курсор также выполняет побайтовое чтение VB и проверки каждой координаты. Это согласуется с замедлением фразового запроса на прогретых данных. Для списков ID картина другая: уменьшение числа читаемых байтов может перевесить затраты на более простое восстановление разностей.

Эти полные проходы являются отдельной нагрузкой. Фразовый поиск может читать лишь часть списков и дополнительно сопоставляет координаты, поэтому времена проходов нельзя складывать для получения времени запроса. Исходные ответы, все повторения и замеры обоих пространств сохранены в `lab11/profile/`; запуск описан в `lab11_run.md`.

5.4 Повторная сборка на расширенном корпусе

На отдельном снимке от 12.11.2026 из 798 999 публикаций получено 3 457 590 термина в двух пространствах. В таблице сопоставлены размеры несжатых файлов, сжатых списков с таблицами смещений и самостоятельной рабочей копии.

Данные	Исходные, МБ	Сжатые, МБ	Экономия
Списки и координаты	6991,76	2016,27	71,2 %
Полный индекс	7441,44	2312,17	68,9 %

Сжатые списки полностью сравнены с исходными: каждый ID и каждая координата совпали. Самостоятельная копия открылась поисковым движком. Для девяти булевых и фразовых запросов в обоих пространствах совпали количество ответов и первые ID исходного, сжатого и packed-вариантов. Отдельный повторный `verify-packed` не запускался. Ранее размеченные 15 документов относятся к старому снимку; их ID не переносились на новый. Сводка повторного измерения сохранена в `lab11/refresh_2026-11-12.json`.

6 Вывод

В этой работе я добавил сжатие словаря, списков документов и координат слов. Полный рабочий индекс удалось уменьшить с 4443,34 до 1400,56 МБ, то есть на 68,5 %. Проверка всех восстановленных данных не выявила отличий от исходного индекса, а на тестовых запросах результаты поиска совпали. На трёх запросах с предварительной разметкой публикаций средняя P@5 сохранилась на уровне 0,60 без падения.

Лучше всего сократились списки документов: вместо полных ID в них хранятся небольшие разности. При этом сжатие не всегда ускоряет поиск. Например, запрос «god of war» по словоформам стал выполняться дольше. Отдельные замеры показали, что сжатые данные читаются быстрее, но восстановление координат требует больше времени. Получился заметный выигрыш по месту, но скорость ещё есть куда улучшать. На расширенном корпусе полный индекс сократился на 68,9 %, а совпадение результатов поиска сохранилось.

1 Постановка задачи

Ускорить булев и цитатный поиск по сжатому индексу, добавив переходы через части списков, которые не могут попасть в результат. Проверить сохранение выдачи, измерить время поиска, количество обработанных записей и дополнительный размер индекса. Реализовать основные структуры и алгоритмы на C++ без STL-контейнеров, подготовить автотесты, консольный запуск и веб-демонстрацию.

2 Метод решения

2.1 Исходный поиск

Используется самостоятельная сжатая копия индекса ЛР11 для 478 833 документов. Индекс содержит пространства словоформ и лемм; числовые списки хранятся как разности, записанные кодом Variable Byte. Словарь терминов использует блочное префиксное сжатие. Форматы этих данных в ЛР12 не изменялись.

При последовательном пересечении двух упорядоченных списков указатель меньшего элемента сдвигается на одну запись. Для частого слова приходится восстанавливать много ID или координат, которые заведомо меньше следующего элемента другого списка. Например, для пересечения $(2, 4, 8, 34, 35, 64, 128)$ с текущим ID 34 можно сразу перейти к 34, не читая промежуточные записи.

2.2 Указатели и состояние VB-декодера

За основу взята эвристика из лекции «Прыжки по индексу» (слайд 22): около $\sqrt{P}$ равномерных указателей для списка длины P. В реализации шаг равен $s = \lceil \sqrt{P} \rceil$, а точки сохраняются после записей $s, 2s, 3s, \ldots$. Для $P < 256$ указатели не создаются: загрузка точек короткого списка может оказаться дороже его последовательного чтения. Порог 256 выбран как простое ограничение накладных расходов и отдельно по корпусу не оптимизировался.

Каждая точка занимает 24 байта и содержит абсолютное значение, байтовое смещение следующего кодового слова внутри списка и число уже прочитанных записей. Для постинга абсолютное значение — полный 64-битный ID. Для координаты оно включает номер документа, поле и позицию токена. Из этих частей восстанавливается состояние для декодирования следующей разности. Нельзя просто перейти к произвольному байту VB-потока: нужны граница числа и предыдущее значение.

Вызов `advance(target)` ищет последнюю точку со значением не больше `target`. Переход разрешён только вперёд от текущей записи. Если значение точки равно цели, оно возвращается сразу; иначе чтение продолжается до первого значения не меньше цели. Точка больше цели не используется, поскольку перед ней может находиться нужное совпадение. При последовательных вызовах не требуется хранить сами списки в памяти: курсор загружает только указатели своего термина.

2.3 Булевы операции и фразы

В пересечении меньший операнд продвигается до текущего значения большего. Этот принцип применяется и к составным операндам. Собственные физические указатели есть только у исходных списков терминов; промежуточные множества остаются потоковыми.

Выражение A AND NOT B вычисляется как разность $A \setminus B$. Раньше сначала строилось потоковое дополнение B относительно всех документов, а затем пересекалось с A. Упрощение сохраняет смысл выражения и устраняет лишний обход таблицы документов. Для самостоятельного NOT полный обход остаётся необходимым. В OR нужно вернуть элементы обоих списков, поэтому пропускать потенциальные результаты нельзя.

В цитатном поиске координата i-го слова сдвигается на i позиций назад. После этого пересекаются сдвинутые координаты. Меньшая группа продвигается к большей с учётом обратного сдвига и границы поля. Несколько лемм одного слова объединяются по исходным координатам. Повторные слова, порядок токенов и раздельность заголовка и текста сохраняются.

2.4 Формат расширения и проверка

Четыре отдельных файла хранят таблицы терминов и точки для постингов и координат. Запись таблицы занимает 16 байт: смещение и число точек. Заголовки содержат версию, размеры, контрольные суммы FNV-64 и отпечаток заголовков сжатого индекса. Перестроение корпуса или другой вариант упаковки требует своих указателей. Незавершённое расширение имеет маркер `INCOMPLETE` и не открывается для поиска.

Команда `skip_index verify` проверяет контрольные суммы и каждую сохранённую точку против декодированного VB-потока. При обычном поиске проверяются заголовки, привязка и границы нужных списков; полная проверка выполняется отдельно после построения или переноса файлов.

3 План и результаты тестирования

1. Сравнение полных векторов ID последовательного и ускоренного поиска для AND, OR, NOT, вложенных выражений и неизвестных терминов.

2. Проверка фраз с повторными словами, нескольких лемм, порядка токенов и границы заголовка и основного текста.

3. Прямые вызовы продвижения к точному значению точки, между точками, через несколько документов и к максимальному 64-битному ID.

4. Проверка пустого индекса, повреждения, усечения, чужой привязки, незавершённой сборки, защиты существующего каталога и резерва диска.

5. Проверка CLI и передачи пути указателей из веб-формы в C++-поиск.

6. Сверка всех точек полного индекса и сравнение режимов на реальных запросах.

Синтетический корпус содержит 600 документов с разреженными 64-битными ID и длинными списками, достаточными для реальных прыжков. Помимо равенства выдачи проверяется уменьшение числа декодированных координат и посещений универсума. Пройдены все 132 проверки CTest и 283 Python-теста; 6 интеграционных проверок MongoDB пропущены. В группе ЛР12 — 9 проверок CTest и 2 Python-теста. Журнал тестов приложен в `lab12/tests.txt`. Веб-форма дополнительно проверена с настоящим индексом в обоих пространствах терминов.

4 Результаты

4.1 Размер и условия эксперимента

Расширение с таблицами занимает 200,56 МБ, то есть добавляет 14,3 % к сжатому рабочему индексу ЛР11. Здесь МБ означает 10^6 байт. Исходные VB-потоки и строки не дублируются.

Сравниваются три режима одного исполняемого файла: `linear` — последовательный алгоритм; `advance` — продвижение до цели и упрощение отрицания без указателей; `skips` — те же оптимизации с указателями. Так отдельно видна польза алгоритмических изменений и физических прыжков.

Выполнено 7 раундов для каждого пространства терминов. В каждом раунде режим запускается в новом процессе, обрабатывает один прогревочный пакет и один измеряемый. Порядок режимов циклически меняется, а порядок запросов разворачивается через раунд. Кэш ОС не очищается. Время включает разбор запроса, загрузку точек нужных терминов и перечисление всех совпадений; открытие индекса, загрузка морфологии и вывод метаданных в него не входят. В таблице приведены медианы, а не время одного запуска.

4.2 Время и объём работы

Результаты для пространства словоформ, время в мс:

Запрос	linear	advance	skips
`"god of war"`	167,14	145,85	114,01
`god AND of AND war`	10,97	8,39	9,00
`doom AND eternal`	0,90	0,98	1,05
`doom AND of`	10,10	7,25	6,88
`игры AND NOT doom`	995,97	8,68	8,79
`NOT doom`	980,02	979,22	951,30
`doom OR eternal`	0,91	0,90	0,96

Фраза «god of war» ускорилась с 167,14 до 114,01 мс, то есть в 1,47 раза. При этом относительно режима `advance` выигрыш меньше: часть ускорения даёт само продвижение до цели без повторных сравнений на каждом уровне выражения.

Число реально декодированных постингов либо координат и выполненных прыжков в тех же запросах:

Запрос	linear	skips	Прыжков
`"god of war"`	2083861	1529296	1509
`god AND of AND war`	262607	238814	576
`doom AND of`	231973	199984	462
`doom AND eternal`	10562	10198	80
`игры AND NOT doom`	156964	154101	9
`NOT doom`	7469	7469	0
`doom OR eternal`	10562	10562	0

Для «god of war» число декодированных координат сократилось примерно на 27 %. Для `игры AND NOT doom` основной выигрыш связан с упрощением выражения: посещения 478 833 записей универсума заменены разностью двух списков. Это не следует приписывать только прыжкам.

На коротком `doom AND eternal` указатели замедляют поиск: пропускается мало записей, а загрузка точек и переходы по файлам требуют времени. У одиночного термина и OR полезных прыжков нет. Самостоятельный NOT также остаётся дорогим, поскольку нужно перечислить дополнение. Небольшие колебания времени таких запросов не доказывают алгоритмического ускорения.

В обоих пространствах для всех 13 запросов и всех повторений совпали число результатов, показанные документы и контрольная сумма полного упорядоченного потока ID. Полные результаты замеров, счётчики и SHA-256 расширения, словаря и программ находятся в `lab12/summary.json` и JSONL-файлах. Счётчики отражают логическое декодирование, а не физические обращения к накопителю. Скорость холодного чтения отдельно не измерялась.

4.3 Указатели на расширенном корпусе

Для нового снимка из 798 999 документов указатели построены заново по самостоятельной сжатой копии индекса. Получено 3 280 689 точек в списках документов и 4 068 765 точек в координатах. При построении декодированы 306 785 548 записей списков и 567 184 357 координат.

Пробный запрос `doom AND eternal` в оптимизированном режиме нашёл 2 401 документ, как и поиск без указателей; первые три ID также совпали. Счётчики оптимизированного варианта: 17 171 декодированный постинг, 86 прыжков и 563 пропущенные записи. Времена и таблицы выше относятся к исходному снимку из 478 833 документов; повторный полный бенчмарк на расширенном корпусе не проводился.

5 Вывод

В этой работе я добавил прыжки по спискам документов и координатам слов. Для фразы «god of war» удалось сократить число декодированных координат и ускорить поиск примерно в полтора раза относительно последовательного варианта. Результаты поиска при этом сохранились.

Самый большой выигрыш на запросе с AND NOT получился благодаря устранению лишнего обхода всего корпуса. При этом указатели помогают не везде: короткие списки иногда выгоднее читать подряд. За ускорение пришлось заплатить дополнительными 200,56 МБ на диске. Получился полезный выигрыш для длинных пересечений, но универсального ускорения всех запросов нет.

1 Постановка задачи

Добавить в поисковую систему ранжирование документов на основе TF-IDF. Вместо порядка по идентификатору выдавать сначала документы с большей оценкой соответствия запросу. Реализовать запросы из обычных слов, ограничение выдачи top-k, интерфейс командной строки и веб-форму. Проверить численную правильность оценок, измерить время и дополнительный объём индекса, рассмотреть изменение первых результатов.

Основные структуры и алгоритмы поиска реализуются на C++ без STL-контейнеров. Python используется для веб-обвязки, запуска экспериментов и подготовки таблиц. Входные и выходные данные имеют кодировку UTF-8.

2 Метод решения

2.1 От булева поиска к весам терминов

Булев поиск определяет, подходит документ запросу или нет. Если результатов много, порядок по ID не помогает выбрать наиболее подходящие публикации. Для ранжирования документ и запрос представлены разреженными векторами терминов. Выбрана схема `lnc.ltc` из лекции о текстовом ранжировании (`lesson-09`, слайды 41–43).

Частота $tf_{t,d}$ равна числу вхождений термина t в документ d. Координаты из ЛР10 позволяют посчитать её без повторного чтения корпуса: вхождения в заголовке и основном тексте складываются. В этой работе поля имеют одинаковый вес. Логарифмическая частота задаётся как

$$L(x) = \begin{cases} 1 + \log_{10} x, & x > 0, \\ 0, & x = 0. \end{cases}$$

Так повторение слова увеличивает вес, но увеличение частоты в десять раз не означает десятикратного увеличения вклада в оценку.

Документная частота df_t — число документов, содержащих термин, а N — число всех документов снимка, включая пустые. Обратная документная частота равна $idf_t = \log_{10}(N/df_t)$. Она отличается от общего числа вхождений слова в корпус. Редкий термин имеет больший IDF; термин с $df_t = N$ имеет нулевой IDF.

2.2 Схема lnc.ltc и косинусная нормировка

В обозначении `lnc.ltc` три символа до точки относятся к документу, три после неё — к запросу. `l` означает логарифмический TF, `n` — отсутствие множителя IDF, `t` — IDF, `c` — косинусную нормировку. Таким образом,

$$w_{t,d} = L(tf_{t,d}), \qquad w_{t,q} = L(tf_{t,q})\, idf_t,$$

$$\|d\| = \sqrt{\sum_{t \in V} w_{t,d}^2}, \qquad \|q\| = \sqrt{\sum_{t \in q} w_{t,q}^2}, \qquad score(d,q) = \frac{\sum_{t \in q \cap d} w_{t,d} w_{t,q}}{\|d\|\,\|q\|}.$$

Норма документа считается по *всему* словарю пространства V. Если ограничить её словами запроса, дополнительные слова документа перестанут влиять на нормировку, и получится другая модель. Оценка является косинусным сходством преобразованных векторов, а не вероятностью релевантности.

В тексте слайда 42 нормировка описана неточно; в реализации используется значение `c` из таблицы на слайде 41 и нормированный пример слайда 43. Словам запроса, отсутствующим в словаре корпуса, не соответствует ось: они исключаются из расчёта. Если после этого норма запроса равна нулю, выдача пуста. Документы с нулевой оценкой не включаются в результат.

2.3 Построение и использование норм

Для каждого термина исходного координатного индекса последовательно читаются его вхождения и группируются по документам. По полученному TF к сумме квадратов соответствующего документа добавляется $L(tf)^2$. Одновременно проверяется совпадение DF с булевым индексом. После полного прохода из сумм извлекаются квадратные корни.

Один физический индекс содержит пространства словоформ и лемм. Нормы для них считаются отдельно. При неоднозначной лемматизации каждая различная альтернатива является отдельным термином, как в предыдущих работах. Если альтернативы нескольких слов запроса совпали, они объединяются в одну ось с суммированием query TF.

Расширение хранит два массива норм double и заголовок 80 байт. В нём записаны версия формулы, размеры, FNV-64 и привязка к исходным заголовкам индекса. Расширение совместимо с исходной и соответствующей упакованной копией. При открытии проверяются привязка, размеры, контрольная сумма и допустимость всех значений. Незавершённая сборка помечается `INCOMPLETE` и не открывается.

2.4 Поиск и отбор top-k

Запрос токенизируется; при необходимости выполняется лемматизация. После объединения одинаковых терминов вычисляются веса и норма запроса. Координаты каждого его термина читаются последовательно. Для очередного документа подсчитывается TF и увеличивается накопленная оценка. Затем оценки делятся на заранее рассчитанные нормы документов.

Накопитель содержит N значений double. Для top-k используется собственная ограниченная двоичная куча: в корне находится худший из отобранных результатов. Лучший кандидат заменяет его с восстановлением кучи. В конце куча сортируется по убыванию оценки; при точном равенстве раньше идёт меньший исходный ID. Поддерживаются произвольные 64-битные ID, включая максимальное значение.

Память основных массивов составляет $16N$ байт норм и $8N$ байт оценок, то есть 11,49 МБ на данном корпусе. Дополнительно нужны буферы, словарь лемматизатора и $O(k)$ памяти результатов. Время состоит из чтения координат терминов запроса, обхода массива $O(N)$ и отбора $O(M \log k)$, где M — число документов с положительной оценкой. Полный словарь и координатный индекс в память не загружаются.

В CLI выбран режим `-ranking tfidf`; запросы читаются по строкам stdin, ответы выдаются в JSONL. Поле `score` содержит оценку, `total` — полное число положительных совпадений, `-limit` ограничивает только число показанных результатов. Веб-форма вызывает тот же C++-поиск и позволяет выбрать способ выдачи. В режиме TF-IDF слова без кавычек образуют «мешок слов». Фрагменты в двойных кавычках проверяются по координатам и обязательны целиком: слова должны стоять подряд, в указанном порядке и в одном поле. После отбора документов их оценки TF-IDF определяют порядок выдачи. Несколько фраз означают совместное выпол-

нение всех фраз; остальные слова влияют на оценку. Булев язык AND/OR/NOT доступен в отдельном режиме.

3 План и результаты тестирования

Для численной проверки используется небольшой корпус с заранее известными частотами. Независимый эталон в GoogleTest строит полные векторы из текстов, вычисляет DF, TF-IDF и косинус, а затем полностью сортирует результаты. Основной алгоритм сравнивается с ним с допуском 10^{-12}.

План включает следующие группы проверок:

1. Однословные и многословные запросы, повторение слов, неизвестные слова, одинаковые частоты и объединение вхождений заголовка и текста.

2. Норма по всему словарю, включая слова вне запроса; влияние лишних слов на итоговую оценку документа.

3. Точное совпадение top-k с префиксом полной сортировки, $k = 0$, равные оценки и большие ID. Отдельный корпус из 1200 документов проверяет кучу для $k = 1, 7, 20, 1000$.

4. Пустой корпус, пустые документы, отсутствие совпадений и запрос только из терминов с нулевым IDF: отсутствие деления на ноль и NaN.

5. Отдельные нормы лемм, неоднозначные разборы и объединение альтернатив в запросе; идентичность оценок исходного и сжатого индексов.

6. Цитаты в TF-IDF: полный порядок и соседство слов, граница полей, несколько обязательных фраз, леммы и ошибки кавычек.

7. Повреждённые, усечённые, чужие и незавершённые нормы; недопустимый UTF-8, ограничения длины, последовательность запросов с восстановлением после ошибки.

8. Передача параметров из веб-формы, UTF-8, вывод оценок и выбор TF-IDF через настоящий HTTP-запрос к тестовому серверу.

Исходный прогон ЛР13 содержал 11 GoogleTest, проверку CLI help и 3 Python-теста. Его регрессия завершилась успешно: 144 CTest и 286 Python-тестов; 6 интеграционных проверок MongoDB были пропущены. Исторический журнал сохранён в `lab13/tests.txt`. После добавления проверки цитаты группа ЛР13 содержит 12 GoogleTest и проверку CLI help. Текущий полный прогон: 155 CTest и 296 Python-тестов, без пропусков; журнал находится в `lab17/tests.txt`.

4 Результаты

4.1 Корпус и условия измерений

Использован тот же снимок из 478 833 игровых публикаций, что в предыдущих работах. В двух пространствах суммарно 2 545 616 терминов и 335 548 564 координаты. Построение норм заняло 51,140 с, включая полную проверку исходных булева

и координатного индексов. Это один запуск, а не медиана. Размер расширения — 7 661 408 байт, около 7,66 МБ.

Поиск выполнялся по упакованному VB-индексу. Для измерения времени в каждом пространстве сравнивались OR с порядком по ID и TF-IDF по тем же словам запроса. OR выбран здесь потому, что для заданных запросов он перебирает тот же набор документов, которым TF-IDF назначает положительную оценку. Для сравнения первых результатов дополнительно выполнен более строгий булев запрос с AND; его число найденных документов показано отдельно. Проведено семь раундов, порядок режимов и запросов чередовался. Каждый процесс выполнял прогревочный пакет и затем измеряемый пакет. Кэш ОС не очищался. Приведены медианы времени внутри C++: нормализация запроса, чтение списков, расчёт оценок и отбор результатов. Открытие индекса, загрузка морфологии и норм, чтение метаданных выдачи и вывод JSON в это время не входят.

4.2 Время поиска

В таблице приведены результаты для словоформ. Для выбранных запросов число кандидатов OR совпало с числом положительных оценок TF-IDF; оно не зависит от ограничения top-10.

Запрос	Кандидатов	OR/ID, мс	TF-IDF, мс
doom eternal	9 170	0.892	5.982
god of war	228 464	15.757	135.160
cyberpunk 2077	8 407	1.062	5.601
baldur gate	7 173	0.936	4.013
игры	149 495	6.066	26.313
игрой	22 588	1.150	4.387
несуществующийтерминxyz	0	0.177	0.170

Ранжирование требует больше работы, чем перечисление булева объединения: нужно восстановить TF из координат и вычислить оценки. Особенно заметна разница на `god of war`, где частое слово `of` порождает длинный список. Поэтому TF-IDF здесь улучшает порядок выдачи ценой времени поиска.

Для лемм запросы «игры» и «игрой» дали одинаковое число кандидатов 230 075; медианы составили 67,103 и 63,765 мс соответственно. По словоформам это разные запросы: 149 495 и 22 588 кандидатов. Все оценки и top-10 сохранялись между повторениями; исходный и сжатый форматы дополнительно сопоставлены в численных автотестах.

4.3 Примеры выдачи и оценка по заголовкам

Для запроса `doom eternal` в TF-IDF первыми стали:

Заголовок	Оценка
Doom Eternal — Мнение	0,81278
Сравнение графики Doom vs Doom Eternal	0,67033
5 подтвержденных оружий в Doom Eternal	0,57472
Doom Eternal — 20 минут захватывающего геймплея	0,54836
Fallout 76 — Новая система перков и отличный DOOM Eternal	0,44219

Более строгий запрос `doom AND eternal` нашёл 1 392 документа против 9 170 документов с положительной оценкой TF-IDF. Среди первых пяти результатов AND по

ID два заголовка прямо называли Doom Eternal, в первых пяти TF-IDF — все пять. Таким образом, оба режима способны найти публикации по игре, но формируют разные первые результаты. Общее число результатов здесь нельзя считать показателем релевантности: AND требует оба слова, а TF-IDF допускает одно совпадение.

Для небольшой иллюстративной оценки взяты три запроса по названиям игр. Помощником вручную размечено объединение первых пяти результатов AND/ID и TF-IDF по видимым заголовкам. Метка 1 означает, что заголовок прямо называет искомую игру, включая русское написание; метка 0 — что тема из заголовка не установлена. Для каждого режима рассчитана доля меток 1 в первых пяти результатах (P@5).

Запрос	AND/ID	TF-IDF
doom eternal	0,40	1,00
god of war	0,80	1,00
cyberpunk 2077	0,60	1,00
Среднее	0,60	1,00

Это оценка соответствия заголовков теме на трёх примерах. Полные тексты не размечались, запросы не составляют представительную независимую выборку. По этим результатам нельзя заключить, что общая точность поиска равна 1. Разметка сохранена вместе с ID и заголовками, поэтому расчёт можно повторить.

На рис. 1 и 2 показан ещё один реальный запрос в пространстве лемм. Булев режим выполняет `чит AND коды AND gta` и находит 6 документов. Режим TF-IDF обрабатывает `чит коды gta` как запрос из слов и находит 18 743 документа с положительной оценкой. В булевой первой пятёрке по ID есть «Gears of War», «Наши истории» и статья о чит-кодах вообще; у TF-IDF первыми идут публикации, в заголовках которых упоминаются GTA и читы или коды. Один документ, ID 431 639, присутствует в обеих выдачах; TF-IDF ставит его на первое место. Скриншоты показывают порядок и число результатов в отдельных запусках веб-интерфейса; время на них не является медианой измерений из предыдущей таблицы.

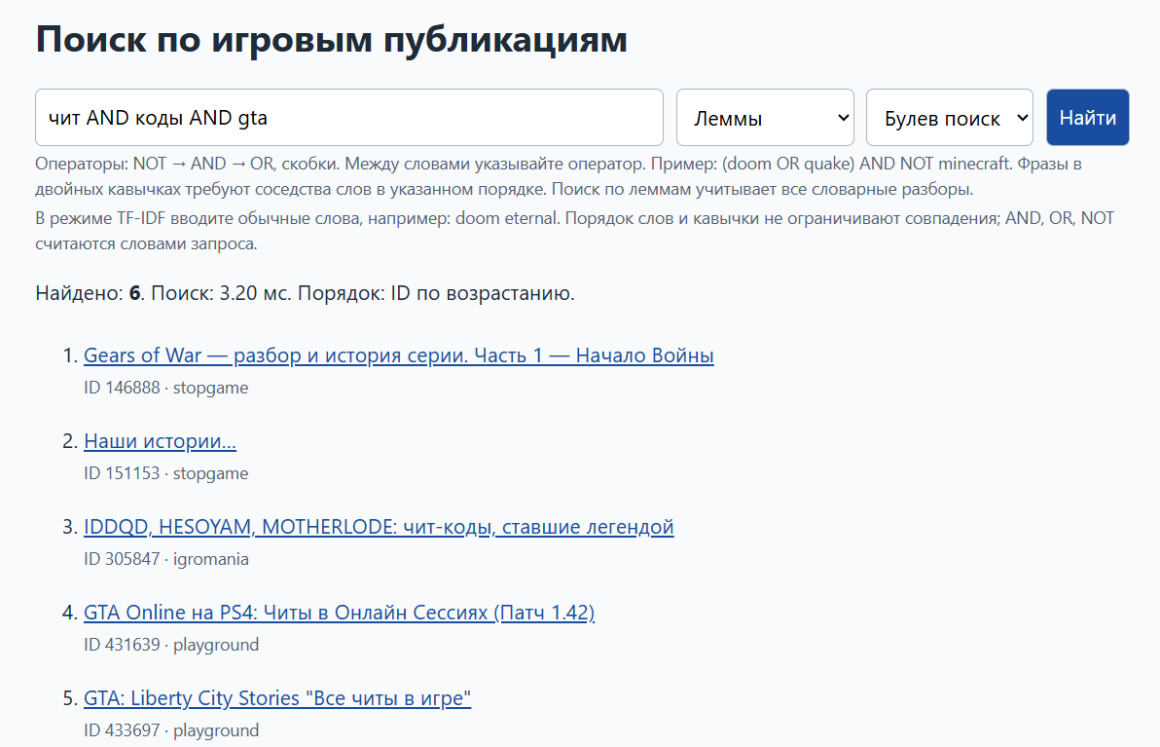


Рис. 15 — Булев запрос по леммам: все три слова обязательны, выдача по ID

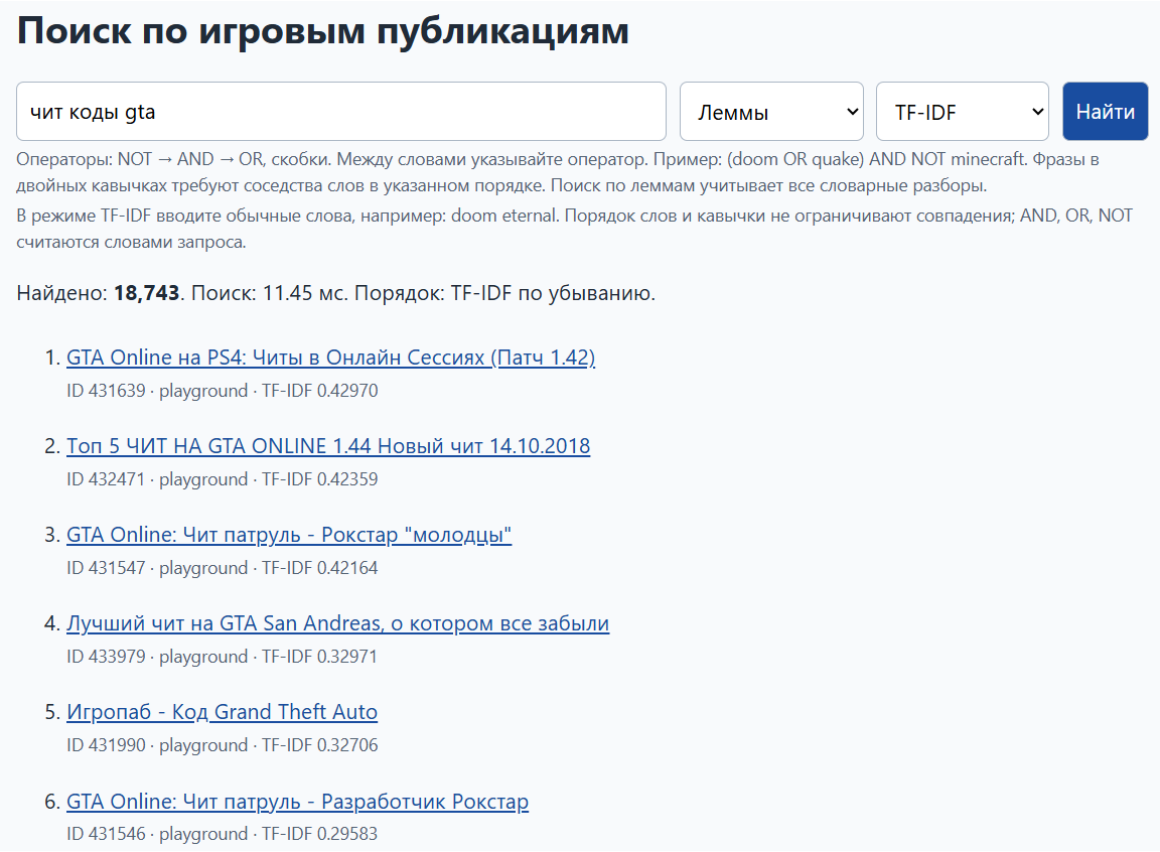


Рис. 16 — Запрос TF-IDF по леммам: 18 743 документа с положительной оценкой

Первые пятёрки TF-IDF в этих примерах состоят из публикаций `Playground`.

Это также требует внимания: косинусная модель может высоко оценивать короткие тексты с большой долей слов запроса. Высокая оценка не означает полноту, актуальность или достоверность материала. Разнообразие источников, качество содержимого и дату эта модель не учитывает.

4.4 Повторное измерение на расширенном корпусе

На новом снимке из 798 999 публикаций индекс содержит 3 457 590 терминов и 567 184 357 координат. Файл норм для обоих пространств занимает 12 784 064 байта. Ранжирование по самостоятельному сжатому индексу повторено для тех же семи запросов; приведены медианы трёх раундов после прогрева. OR и TF-IDF перебирают одинаковых кандидатов, но показывают их в разном порядке.

Запрос	Кандидатов	OR/ID, мс	TF-IDF, мс
doom eternal	15 334	2.431	12.761
god of war	375 438	32.669	359.248
cyberpunk 2077	14 255	2.447	16.367
baldur gate	12 091	1.789	10.005
игры	255 589	18.642	61.673
игрой	38 489	2.635	8.549
несуществующийтерминхуz	0	0.384	0.276

Например, `doom eternal` даёт 15 334 кандидата с положительной оценкой TF-IDF, а более строгий булев запрос `doom AND eternal` — 2 401 документ. Скриншоты и оценка P@5 выше относятся к старому снимку из 478 833 документов. Его ID и метки на новые результаты не переносились; независимая оценка релевантности на расширенном корпусе не проводилась. Ответы и сводка сохранены в `lab13/refresh_2026-11-12.json`.

4.5 Обязательная цитата при ранжировании

После замечания на защите фраза в кавычках стала условием включения документа и в режиме TF-IDF. Ранее кавычки удалялись при токенизации, поэтому после первого точного результата могли идти документы с отдельными словами цитаты. Теперь координатный поиск проверяет каждую фразу для кандидата до подсчёта итогового числа и отбора top-k.

На новом снимке запрос `doom eternal` по словоформам даёт 15 334 документа с положительной оценкой. Запрос с кавычками `"doom eternal"` оставляет 2 210 документов, столько же, сколько отдельный булев поиск этой фразы. Пять первых ID ранжированной выдачи: 719185, 719211, 719213, 719178 и 719182; все проходят проверку соседних токенов в одном поле. По леммам фраза `"игры ps plus"` даёт 4 071 документ в обоих режимах. Эти числа получены одним запуском и не являются сравнением времени: проверка цитаты добавляет чтение координат. JSON-ответы сохранены в `lab13/phrase_filter_2026-11-12.json`.

5 Вывод

В этой работе я добавил ранжирование TF-IDF по схеме `lnc.ltc`. Поисковая система принимает обычные слова и возвращает top-k документов с числовыми

оценками. Независимые автотесты подтвердили расчёт косинуса, порядок выдачи и одинаковую работу с исходным и сжатым индексами.

На рассмотренных запросах TF-IDF поднял тематические публикации в первые строки выдачи даже при сравнении с более строгим булевым AND. При этом поиск стал дороже: например, для `god of war` медиана увеличилась с 15,757 до 135,160 мс. Нормы потребовали всего 7,66 МБ, но извлечение частот из длинных координатных списков занимает время. Его можно уменьшить, если сохранять TF непосредственно в постингах.

Для слов без кавычек модель не учитывает порядок. Также не учитываются разные веса полей и контекст неоднозначных лемм. В выдаче заметно преимущество коротких публикаций; небольшая оценка по заголовкам не заменяет разметку полных документов. Дальше имеет смысл расширить набор запросов, сравнить модели нормировки и отдельно исследовать влияние длины текста на результат.

1 Постановка задачи

Для найденных документов построить короткие фрагменты исходного текста, объясняющие совпадение с запросом. Показывать сниппеты в командной строке и веб-выдаче. Результаты поиска и их порядок должны сохраняться.

2 Метод решения

2.1 Доступ к исходному тексту

Поисковый индекс ЛР7–ЛР13 хранит заголовки и координаты, но не тела документов. Тексты находятся в неизменяемом снимке MAIIR3 из 478,833 документов. Для быстрого доступа создан отдельный файл смещений: для каждого ID записаны положение и длина заголовка и текста. Файл создаётся однократно потоковым проходом по снимку и занимает 19,153,352 байта; тела документов в оперативную память при построении не загружаются. При запросе читаются только тексты показанных документов.

Файл смещений содержит версию формата, размер снимка, число документов и контрольную сумму записей. При открытии проверяются границы и порядок ID. Заголовок каждого показанного документа сравнивается побайтово с заголовком из поискового индекса. Это защищает от случайной подмены снимка или смещений. При отсутствии файла CLI может восстановить смещения при открытии, но такой путь не подходит для частых запросов.

2.2 Выбор фрагмента

Исходный текст делится на предложения по точке, вопросительному и восклицательному знакам перед пробелом, а также по переводу строки. Токенизация использует те же правила `gaming-v3`, что и индекс. В режиме лемм варианты терминов сопоставляются через тот же словарь. Для предложения подсчитываются число разных слов запроса и число совпадений. Компактное расположение совпадений повышает оценку; при равенстве предпочтение отдаётся более раннему предложению. Если в тексте нет слов запроса, показывается его начало.

После выбора предложения сниппет расширяется соседними предложениями в пределах того же абзаца. Это даёт контекст коротким предложениям и не захватывает соседние блоки рекомендаций сайта. Фрагмент ограничивается 600 байтами, по возможности вокруг первого совпадения. Обрезка выполняется только на границе символа UTF-8. Ответ CLI содержит сам текст, байтовые диапазоны совпадений внутри него и признаки обрезки слева и справа. Веб-форма преобразует эти диапазоны в подсветку `<mark>` после HTML-экранирования фрагмента. Текст сниппета выводится меньшим, приглушённым серым шрифтом; совпавшие слова остаются контрастными. Код выбора и хранения смещений написан на C++ без STL-контейнеров.

3 План тестирования

Случай	Ожидаемый результат
Несколько предложений	Выбирается лучшее, добавляются соседи из того же абзаца
Русский UTF-8	Диапазоны выделяют полные слова в байтовой строке
Прямое чтение и файл смещений	Сниппеты и порядок результатов одинаковы
Повреждённый файл смещений	Ошибка контрольной суммы до выдачи документов
Текст отсутствует	Пустой сниппет без ошибки поиска
HTML в тексте документа	Теги экранированы; подсвечены только слова запроса
Реальный снимок и веб-форма	CLI возвращает фрагменты, HTTP-ответ содержит подсветку

Семь GoogleTest вызывают C++-поиск на синтетическом корпусе: проверяют выбор предложения, точность UTF-8 диапазонов, совпадение двух способов чтения, отказ на повреждении, привязку снимка к индексу, обрезку длинного предложения, пустой текст и фильтрацию до ограничения выдачи. Три Python unittest проверяют передачу параметров ядру и безопасное отображение HTML. Весь набор предыдущих лабораторных также запущен на текущих исходниках.

4 Результаты

4.1 Проверка на полном снимке

Файл смещений успешно построен для 478,833 документов. Скрипт `run_lab14.py` выполнил пять запросов на упакованном индексе: TF-IDF по словоформам и леммам, а также булев поиск. Для запроса `doom AND eternal` булев поиск нашёл 1,392 документа; для первого результата выбран фрагмент о выпуске Doom Eternal. Для TF-IDF запроса `doom eternal` найдено 9,170 документов; первые пять результатов и сниппеты сохранены в `lab14/examples.json`. HTTP-запрос к веб-форме вернул код 200, на сохранённой странице `lab14/web_example.html` есть 27 подсвеченных вхождений. Для запроса `"игры ps plus"` AND `сентябрь` по леммам найдено шесть документов. У первых двух короткое предложение со словами запроса расширилось до полного абзаца: 559 и 542 байта вместо одной строки. У третьего документа слова встречаются только в блоке «Читайте также»; переходить в соседний абзац здесь означало бы показывать чужой контекст.

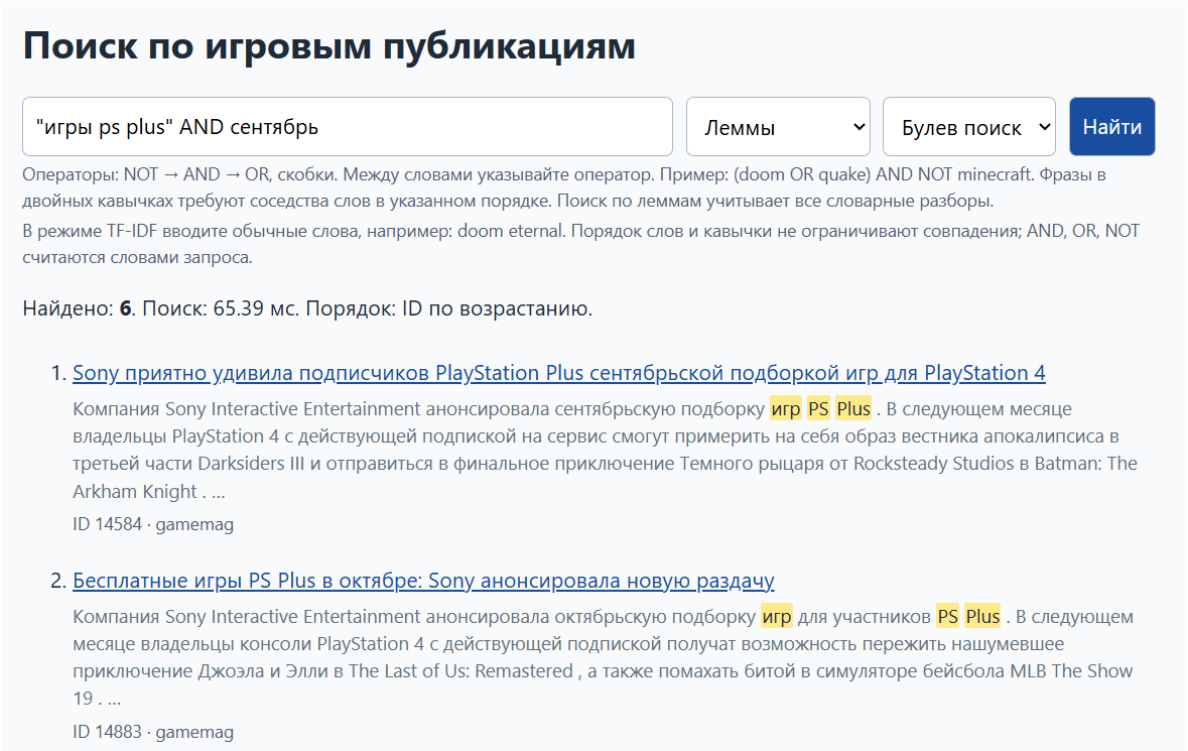


Рис. 17 — Булев поиск по леммам: запрос `"игры ps plus" AND сентябрь`; два первых результата с расширенными сниппетами

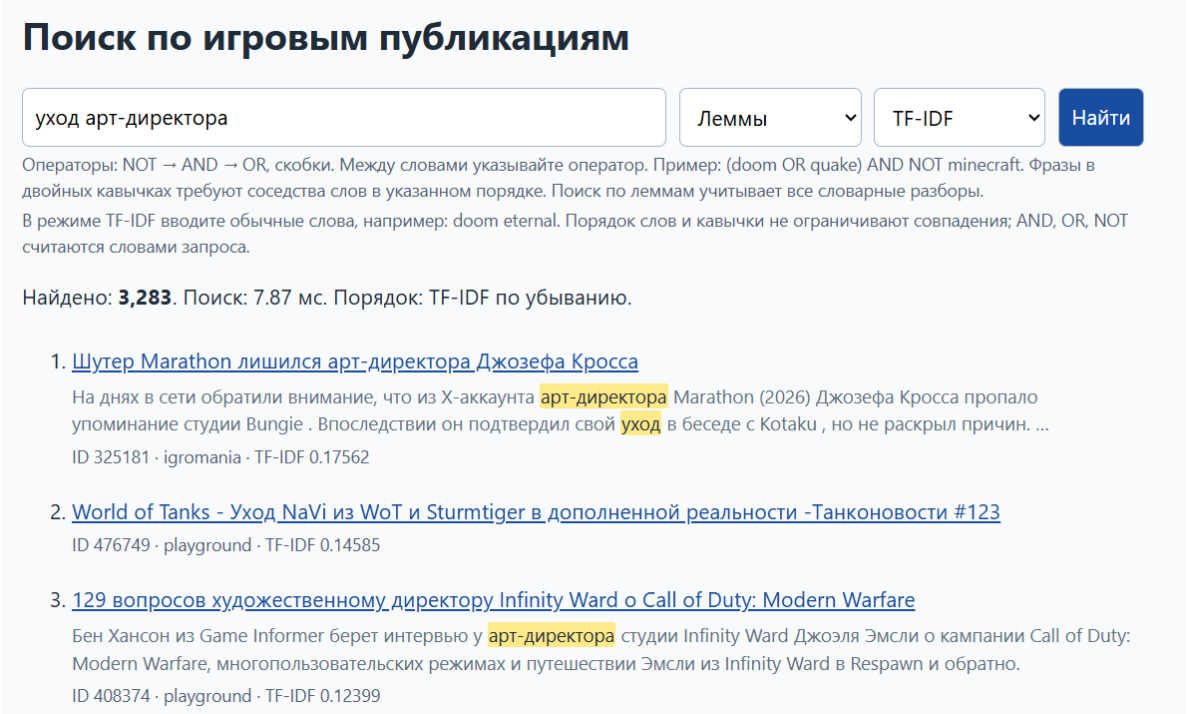


Рис. 18 — TF-IDF по леммам: запрос «уход арт-директора»; у второго результата текст документа отсутствует, поэтому сниппет пуст

С файлом смещений полный вызов CLI для запроса `doom eternal` занял 0,149 с. Первый вариант, просматривавший снимок при каждом запросе, требовал около 9 с. Это наблюдения одного запуска при тёплом кэше, включая открытие индекса и подготовку выдачи, а не гарантированное время ответа на другом компьютере.

4.2 Проверка на расширенном корпусе

Для нового снимка из 798 999 документов построен отдельный файл смещений размером 31 959 992 байта. На том же снимке повторены пять CLI-запросов и HTTP-проверка веб-формы. Например, TF-IDF запрос `doom eternal` даёт 15 334 документа с положительной оценкой, булев `doom AND eternal` — 2 401 документ, а фразовый запрос `"игры ps plus"AND сентябрь` по леммам — 10 документов. Для всех пяти запросов получены сниппеты из нового корпуса. HTTP-запрос вернул код 200; на странице обнаружено 34 подсвеченных вхождения. Числа и скриншоты предыдущего подраздела относятся к исходному снимку из 478 833 документов. Новые ответы и SHA-256 входов сохранены в `lab14/refresh_2026-11-12.json`.

4.3 Сниппеты для обязательной цитаты в TF-IDF

В ранжированном режиме фраза `"doom eternal"` теперь проверяется по координатам до включения документа в выдачу. На новом снимке она даёт 2 210 результатов вместо 15 334 у запроса без кавычек `doom eternal`. Для первых трёх результатов получены ID 719185, 719211 и 719213. У документа 719211 основной текст отсутствует, поэтому сниппет пуст, но сама фраза находится в индексируемом заголовке. Проверка цитаты меняет отбор документов; генерация сниппета по-прежнему показывает контекст уже найденного документа. Ответ сохранён в `lab14/phrase_filter_2026-11-12.json`.

4.4 Автотесты

Текущие тесты ЛР14: 7/7 GoogleTest и 3/3 Python unittest. Полный регрессионный прогон: 155/155 CTest и 296/296 Python-тестов, без пропусков; для интеграционных проверок запущена отдельная временная MongoDB. Текущий журнал и сводка находятся в `lab17/`; исходный прогон ЛР14 сохранён в `lab14/full/`.

5 Вывод

Сниппеты делают причину совпадения видимой прямо в выдаче и работают поверх обоих режимов поиска без изменения ранжирования. Для документов, у которых в снимке сохранён только заголовок, текстового фрагмента нет. Независимо от этого короткие предложения могут почти повторять заголовок. Простое правило границ предложений иногда ошибается на сокращениях, а ограничение в 600 байт может отрезать полезный контекст. В дальнейшем можно учитывать совпадение с заголовком и тип публикации, а также лучше отделять основной текст от рекомендаций сайта.

1 Постановка задачи

Подготовить воспроизводимый набор автоматических проверок для компонентов учебной поисковой системы. Обосновать тестовый план, покрывающий основные правила обработки документов, построения индекса и выполнения запросов.

2 Метод решения

Основные C++-компоненты проверяются GoogleTest. Тесты токенизатора вызывают его непосредственно; индекс и поиск проверяются на небольших синтетических корпусах с заранее известными результатами. Тесты Python unittest проверяют правила робота, HTML-парсеры, экспорт, веб-обвязку и обработку ошибок. Модульные тесты используют небольшие входы с известным результатом. Интеграционные тесты запускаются с отдельной временной MongoDB.

Метаданные фильтров хранятся отдельно от текстового индекса. Для каждого ID сохраняются сайт, тип и год публикации. Год извлекается из даты статьи; если дата отсутствует или текущая запись MongoDB отличается от снимка, год остаётся неизвестным. Поиск применяет фильтр до ограничения выдачи, поэтому общее число результатов и top-k считаются по отобранным документам.

Общий запуск выполняет `run_tests.py --lab all`: он пересобирает C++-тесты в Docker, выполняет CTest по отдельным именованным случаям, после чего запускает Python unittest в подробном режиме. Скрипт `run_lab17.py` сохраняет полный стандартный вывод и ошибки в `lab17/tests.txt`, запись о тестовом образе в `lab17/build.txt`, журнал временной MongoDB в `lab17/mongo.txt` и сводку с контрольными суммами в `lab17/summary.json`. Если образа нет, он собирается из Dockerfile; если есть, используется существующий. C++-программы при каждом прогоне пересобираются из текущих исходников. Тестовая MongoDB запускается в отдельном контейнере без постоянного тома и останавливается после прогона.

Для Variable Byte использован отдельный эталонный кодировщик, реализованный в тесте по определению формата. На граничных значениях 0, 127, 128, 16383, 16384, 2^{32}, 2^{63} и $2^{64} - 1$ проверяется восстановление числа из файла и из памяти. Декодер должен остановиться ровно после текущего числа и не поглотить начало следующего. Каждый неполный префикс десятибайтовой записи максимального числа должен быть отвергнут.

Для токенизатора проверено свойство объединения независимых полей. Если между двумя корректными полями вставить разделитель, последовательности терминов должны соединиться без изменений; позиции и байтовые границы сдвигаются на известное число токенов и байтов. В проверке есть русские слова, игровые имена, версия и C++/C#.

Тесты Python проверяют правила обработки URL и метаданных. Канонизация удаляет рекламные параметры, сохраняя значимую пагинацию, и повторный вызов не меняет URL. Перенаправление StopGame применяется только к числовому ID новости. Политика исключения отличает страницу категории от вложенной публикации. В JSON-LD находится вложенная публикация; повреждённый блок не скрывает следующий корректный.

3 План тестирования

План составлен по поведению компонентов и возможным отказам. Для каждой группы указан независимый ожидаемый результат, а не только отсутствие аварийного завершения.

Компонент	Проверяемое свойство	Ожидаемый результат
Токенизатор	UTF-8, игровые имена, границы, соединение полей	Точные токены и байтовые смещения; ошибка на повреждённом UTF-8
Лемматизатор	Формы, неоднозначность, неизвестные слова	Все словарные варианты без выдуманных лемм
Индекс	Постинги, координаты, пустые документы и большие ID	Уникальные возрастающие списки, целостный формат
Поиск	AND/OR/NOT, фразы, порядок и соседство	Совпадение с таблицей истинности и независимой проверкой фразы
Сжатие	VB и контрольные точки	Точное восстановление; отказ на усечении и чужом индексе
Ранжирование	TF-IDF, нормировка, top-k, цитаты	Совпадение с независимой формулой, полной сортировкой и проверкой целой фразы
Фильтры	Сайт, тип, год; неизвестная дата; совместный выбор	Точное число совпадений, корректный top-k и сохранение выбора в форме
Робот	URL-категории, метаданные и ошибки	Статьи сохраняются, каталоги отсеиваются, неверные данные не принимаются
Интерфейсы	CLI и веб-форма	UTF-8 и корректный ответ после ошибочного запроса

В наборе использованы фикстуры реальных публикаций для токенизатора, небольшой словарь для морфологии, таблицы истинности для булева поиска, собственное скользящее окно для проверки фраз и прямой расчёт косинуса для TF-IDF. Это позволяет находить ошибки в разных частях системы, не сравнивая программу только с её же повторным запуском.

4 Результаты

4.1 Запуск тестового плана

Команда `python run_lab17.py` выполнена из корня проекта. Готовый Docker-образ использован повторно; исходники C++ собраны заново. Тестовая MongoDB запущена отдельно и остановлена после проверок.

Набор	Всего	Успешно	Пропущено
CTest: GoogleTest и проверки CLI	155	155	0
Python unittest	296	296	0

Все проверки завершились без ошибок. Шесть интеграционных тестов работали с временной MongoDB; каждый использовал отдельную базу со случайным именем. Журнал содержит 155 отдельных строк CTest с именами проверок и результатами, а также имена и результаты 296 Python-тестов. Сводка сохраняет результаты и контрольные суммы файлов запуска.

Случаи выявляют определённые классы регрессий. Например, неверная обработка границы 127/128 или усечённого VB-кода нарушит тест декодера; потеря байтовой границы после соединения полей нарушит свойство токенизатора. Ошибочное обобщение правила каталога PC Gamer приведёт к отбрасыванию вложенной статьи, что покажет тест URL-политики. Тест фильтров проверяет совместный выбор сайта, типа и года на маленьком индексе: первый результат находится после неподходящих документов, а число совпадений не зависит от лимита выдачи. Проверка веб-формы передаёт все три значения в поиск и сохраняет выбранные пункты. Проверка каталога подтверждает, что год загрузки не подменяет отсутствующую дату статьи и что дата изменившегося документа не переносится в старый снимок.

Малые фикстуры не воспроизводят все сбои большого корпуса и внешних сайтов. Однако они позволяют точно проверить поведение компонентов на известных входах.

4.2 Проверка на пополненном корпусе

После основного прогона тестов собран новый неизменяемый снимок из 798 999 публикаций с 2 445 438 731 байтом заголовков и текста. Его индексы проверены отдельно от малых фикстур: полный проход по булеву и координатному индексам, затем сравнение исходного и сжатого sidecar по спискам ID и координатам. Для девяти запросов в двух пространствах терминов совпали число ответов и первые ID исходного, sidecar и packed вариантов. Отдельный полный проход `verify-packed` не выполнялся; поисковое ядро смогло прочитать packed-копию и ответить на эти запросы.

Эти проверки подтверждают согласованность построенных файлов для одного снимка и не входят в число 155 CTest и 296 Python-тестов в таблице выше. Дополнительный тест TF-IDF исключает документы с отдельными словами цитаты, обратным порядком и совпадением через границу полей. На полном индексе `"doom eternal"` даёт 2 210 документов в ранжированном и булевом режимах против 15 334 у запроса TF-IDF без кавычек. Входы, размеры и ограничения проверки записаны в `corpus_refresh_2026-11-12.md`. Поскольку ID в новом снимке назначены заново, старые метки релевантности по ID к нему не применяются.

5 Вывод

В этой работе я собрал единый проверяемый план автотестов поисковой системы. Тесты охватывают главные алгоритмы и пограничные случаи кодирования чисел, токенизации, обработки URL и чтения метаданных. Интеграционные проверки робота выполнены на отдельной временной MongoDB. Полный прогон завершился успешно: 155 проверок CTest и 296 Python-тестов.

Теперь результат можно повторить одной командой и проверить по именованному журналу. Малый набор фикстур не отражает все состояния большого корпуса; по обнаруженным сбоям следует добавлять новые случаи.

Исходный код



Рис. 19 — Исходный код проекта в ветке main

https://github.com/kirbasss/MAI-IR/tree/main